

AW

Aplicaciones Web

UNIDAD 02
CLASE 06

CSS



| CSS

| Los alumnos comprenderán el uso de estilos para realizar efectos que llaman la atención del navegante

| Los alumnos comprenderán el manejo de las pseudoclasas y pseudoelementos para manipular objetos completos o parte de ellos dentro de los estilos



| Los alumnos desarrollaran la comprensión de los beneficios que trae a los desarrolladores web el uso de estilos externos y la ventaja que traen estos a los proyectos profesionales.

| Los alumnos comprenderán los tópicos expuestos hasta el momento desarrollando ejercicios detallados paso a paso para incrementar la habilidad y experiencia con HTML5 y CSS3

ISSD

-Des-

Desarrollo de
Software

MÓDULO

DIDÁCTICO



Desempeño de Exploración

Los alumnos codifican los ejercicios desarrollados en esta clase con ellos exponen inquietudes y dudas que surgen a partir de la codificación y visualización de los mismos.



Desempeño de Investigación

Se arman grupos de investigación para ampliar conocimiento. Los alumnos se muestran los trabajos realizados comentan y dan opinión sobre los mismos. Se exponen trabajos explicando los estilos utilizados.

Introducción a la Clase

Bienvenido a la clase!!, el objetivo de la misma es trabajar con Estilos avanzados, pero aplicándolos en formato externo, es decir a través de un archivo con extensión .css que enlazaremos a cada una de las paginas para que tomen el aspecto deseado. Terminaremos esta clase viendo un montón de ejercicios comentados y detallados para que profundices el uso de estilos y tus proyectos sean cada vez más profesionales!!

Como siempre te deseo, lo mejor para esta clase

Pseudo-clases

Las **:pseudo-clases** tienen muchas cosas en común con los selectores de atributos. Con ellas podremos apuntar a determinados elementos de nuestro código HTML sin la necesidad de emplear clases ni identificadores.

No debemos olvidar algunas diferencias entre **:pseudo-clases** y **::pseudo-elementos**.

| **Las pseudo-clases:** Seleccionan elementos HTML que pueden tener añadidas diferentes clases. Se pueden utilizar varias en un mismo elemento.

| **Los pseudo-elementos:** Seleccionan partes del documento que no son elementos HTML con lo cual no se les pueden aplicar clases. Únicamente admiten un pseudo-elemento por cada selector, éste debe ir colocado al final.

Las :pseudo-clases se escribirán con dos puntos delante **: delante ‘ del nombre ‘** y los pseudo-elementos con doble dos puntos **:: delante ‘ también del nombre**. A continuación veremos algunas:

:root

Selecciona el elemento HTML por ser la raíz del documento.

Aplica los estilos definidos a todo el documento.

```
:root{
  font-family:Arial, Helvetica, sans-serif;
  background-color:#666;
  color:#ccc;
}
```

:nth-child()

Con esta pseudo-clase podremos seleccionar mediante la posición, elementos hijos de una forma alterna. Con este ejemplo vamos a seleccionar del **elemento padre div** el hijo situado en la **segunda posición** y aplicar un estilo.

```
#ejemplo :nth-child(2){    color:#0CF;}
```

:nth-last-child()

Con esta pseudo-clase podremos seleccionar mediante la posición, elementos hijos de una forma alterna. **Siendo la posición inicial el último elemento hijo.**

```
#ejemplo :nth-last-child(3){    color:#0CF;
  text-transform:uppercase;}
```

:first-child

Con esta pseudo-clase seleccionamos el primer elemento hijo.

```
p:first-child{ text-transform:uppercase; color:#CF0;}  
<p>P es un párrafo de nuestro documento - Posición 1</p>  
<p>P es un párrafo de nuestro documento - Posición 2</p>  
<p>P es un párrafo de nuestro documento - Posición 3</p>  
<p>P es un párrafo de nuestro documento - Posición 4</p>  
<p>P es un párrafo de nuestro documento - Posición 5</p>
```

:last-child

Con esta pseudo-clase seleccionamos el último elemento hijo.

```
p:last-child{text-transform:uppercase; color:#CF0;}  
<p>P es un párrafo de nuestro documento - Posición 1</p>  
<p>P es un párrafo de nuestro documento - Posición 2</p>  
<p>P es un párrafo de nuestro documento - Posición 3</p>  
<p>P es un párrafo de nuestro documento - Posición 4</p>  
<p>P es un párrafo de nuestro documento - Posición 5</p>
```

Como en los primeros ejemplos, también podemos hacer referencia al último hijo de un determinado elemento.

```
#ejemplo p:last-child{text-transform:uppercase; color:#9FF;}

<h1>H1 es un título de nuestro documento</h1>
<h2>H1 es otro título de nuestro documento</h2>
<div>
  <p>P es hijo de su padre DIV</p>
</div>
<div id="ejemplo">
  <p>P es hijo de su padre DIV con indentificado "ejemplo"</p>
  <p>P es hijo de su padre DIV con indentificado "ejemplo"</p>
</div>
```

:only-child

Esta pseudo-clase selecciona cuándo hay un único hijo en ese elemento.

```
a:only-child{
  text-decoration:none;
  font-weight:bold;
  color:#6FF;
}
<div>
  < a href="">Enlace 1 del primer DIV< /a>
  < a href="">Enlace 2 del primer DIV< /a>
</div>
```

:nth-of-type()

Con esta pseudo-clase podremos seleccionar mediante la posición, elementos hijos de una forma alterna pero de un **determinado tipo**.

```
#ejemplo p:nth-of-type(2){
  color:#CF0;
}
```

:nth-last-of-type()

Con esta pseudo-clase podremos seleccionar mediante la posición elementos hijos de una forma alterna pero de un **determinado tipo**. Siendo la **posición inicial el último elemento hijo**.

```
#ejemplo p:nth-last-of-type(4){
  font-weight:bold;
  color:#FC0;
}
```

:first-of-type

Con esta pseudo-clase seleccionamos al primer elemento hijo de un determinado tipo.

```
#ejemplo p:first-of-type{
  text-transform:uppercase;
  font-weight:bold;
  color:#9FF;
}
<h1>H1 es un título de nuestro documento</h1>
<h2>H2 es otro título de nuestro documento</h2>
<div>
  <p>P es hijo de su padre DIV</p>
</div>
<div id="ejemplo">
  <p>P es hijo de su padre DIV con indentificado "ejemplo"</p>
  <p>P es hijo de su padre DIV con indentificado "ejemplo"</p>
</div>
```

:only-of-type

Esta pseudo-clase selecciona cuándo hay un único hijo de ese tipo en ese elemento de un mismo tipo.

```
a:only-of-type{
  text-decoration:none;
  font-weight:bold;
  color:#6FF;
}
<div>
  < a href="">Enlace 1 del primer DIV< /a>
  < a href="">Enlace 2 del primer DIV< /a>
</div>
<div>
  <h1>Esto es un título para mi documento</h1>
  <h1>Esto es otro título para mi documento</h1>
  < a href="">Enlace único del DIV 2< /a>
</div>
<div>
  < a href="">Enlace 1 del DIV 3< /a>
  < a href="">Enlace 2 del DIV 3< /a>
</div>
```

:empty

Selecciona un elemento que no tiene hijos. Se considera como hijo también al texto. Es decir, que un elemento que contenga un espacio en blanco no está vacío, por lo tanto no se aplicarán los estilos declarados.

```
:empty{height:100px; width:100px;
  background-color:#CFF; }
<div></div>
<div> </div>
<div>Este div contiene texto</div>
```

Estilos Externos

En este caso, todos los estilos CSS se incluyen en un archivo de tipo CSS que las páginas HTML enlazan mediante la etiqueta `<link>`. Un archivo de tipo CSS no es más que un archivo simple de texto cuya extensión es .css Se pueden crear todos los archivos CSS que sean necesarios y cada página HTML puede enlazar tantos archivos CSS como necesite.

Este archivo contendrá las reglas de estilo (igual como las hemos visto hasta ahora) pero estarán separadas del archivo HTML.

| La ventaja fundamental es que con esto podemos aplicar las mismas reglas de estilo a parte o a todas las páginas del sitio web. Veremos que esto será muy provechoso cuando necesitemos hacer cambios de estilo (cambiando las reglas de estilo de este archivo estaremos cambiando la apariencia de múltiples páginas del sitio).

| También tiene como ventaja que al programador le resulta más ordenado tener lo referente a HTML en un archivo y las reglas de estilo en un archivo aparte.

| Otra ventaja es que cuando un navegador solicita una página, se le envía el archivo HTML y el archivo CSS, quedando guardado este último archivo en la caché de la computadora, con lo cual, en las sucesivas páginas que requieran el mismo archivo de estilos, ese mismo archivo se rescata de la caché y no requiere que el servidor web se lo reenvíe (ahorrando tiempo de transferencia).

Si se quieren incluir los estilos del ejemplo anterior en un archivo CSS externo, se deben seguir los siguientes pasos:

- 1| Se crea un archivo de texto y se le añade solamente los estilos que antes definíamos en la zona de **<style>**
- 2| Se guarda el archivo de texto con el nombre **estilos.css** por ejemplo. Se debe poner especial atención a que el archivo tenga extensión **.css** y no **.txt**
- 3| En la página HTML se enlaza el archivo CSS externo mediante la etiqueta **<link>**:

```
<!DOCTYPE html>
<head>
<link rel="stylesheet" type="text/css" href="/css/estilos.css" media="screen" />
</head>
<body>
<p>Aquí se escribe el texto al que queremos aplicarle el estilo por ejemplo<p>
</body>
</html>
```

Normalmente, la etiqueta `<link>` incluye cuatro atributos cuando enlaza un archivo CSS:

| **rel:** indica el tipo de relación que existe entre el recurso enlazado (en este caso, el archivo CSS) y la página HTML. Para los archivos CSS, siempre se utiliza el valor **stylesheet**

| **type:** indica el tipo de recurso enlazado. Sus valores están estandarizados y para los archivos CSS su valor siempre es **text/css**

| **href:** indica la URL del archivo CSS que contiene los estilos. La URL indicada puede ser relativa o absoluta y puede apuntar a un recurso interno o externo al sitio web.

| **media:** indica el medio en el que se van a aplicar los estilos del archivo CSS. Más adelante se explican en detalle los medios CSS y su funcionamiento.

Imagen 1



Reglas de importancia en los estilos

Los estilos se heredan de una etiqueta a otra, como se indicó anteriormente. Por ejemplo, si tenemos declarado en el **<BODY>** unos estilos, por lo general, estas declaraciones también afectarán a etiquetas que estén dentro de esta etiqueta, o lo que es lo mismo, dentro de todo el cuerpo.

En muchas ocasiones más de una declaración de estilos afecta a la misma porción de la página. Siempre se tiene en cuenta la declaración más particular. Pero las declaraciones de estilos se pueden realizar de múltiples modos y con varias etiquetas, también entre estos modos hay una **jerarquía** de importancia para resolver conflictos entre varias declaraciones de estilos distintas para una misma porción de página. Se puede ver a continuación esta jerarquía, primero ponemos las formas de declaración más generales, y por tanto menos respetadas en caso de conflicto:

- | Declaración de estilos con fichero externo. (Para todo un sitio web)
- | Declaración de estilos para toda la página. (Con la etiqueta **<STYLE>** en la cabecera de la página)
- | Estilos definidos en una parte de la página. (Con la etiqueta **<DIV>**)
- | Definidos en una etiqueta en concreto. (Utilizando el atributo **style** en la etiqueta en cuestión)
- | Declaración de estilo para una porción pequeña del documento. (Con la etiqueta ****)

El atributo opcional **MEDIO**, especifica el medio o medios en que debería aplicarse la hoja de estilo. Entre los valores posibles son:

Descriptor de tipo de medios	Aplicar reglas de hojas de estilo a
"all"	Todos los dispositivos
"aural"	Sintetizadores de voz
"Braille"	Dispositivos táctiles y de braille
"embossed"	Impresoras de páginas de braille
"handheld"	Dispositivos de mano (como PDA
"print"	Medios de páginas (como una página impresa) o páginas que se muestran en pantalla (como una "presentación preliminar").
"Projection"	Proyectores y dispositivos de impresión en diapositivas y transparencias
"screen"	Pantallas en color de computadoras
"tty"	Dispositivos con cuadrículas de caracteres de punto fijo, como teletipos
"tv"	Dispositivos del estilo de la televisión (baja resolución, color, desplazamiento limitado, dispone de sonido).

Archivos Externos

También podemos incluir estilos en un archivo externo con un código con la sintaxis **@import url("estilo.css")**. Se utiliza para definir estilos comunes cuando hay también definición de estilos específicos.

Veamos ahora otra manera de importar una declaración externa de estilos CSS: **@import url("archivo_a_importar.css")**, que se utiliza para importar unas declaraciones de estilos guardadas en la ruta que se indica entre paréntesis. (Las comillas son opcionales, pero los paréntesis son obligatorios, por lo menos, en Explorer).

Se debe incluir en la declaración de estilos global a una página, es decir entre las etiquetas **<style type="text/css">** y **</style>**, que se colocan en la cabecera del documento.

Es importante señalar que la sentencia de importación del archivo CSS se debe escribir en la primera línea de la declaración de estilos, algo parecido al código siguiente.

```
<style type="text/css">
@import url ("estilo.css");
body{
    background-color:#ffffcc;
}
</style>
```

El funcionamiento es el mismo que si escribiésemos todo el fichero a importar dentro de las etiquetas de los estilos, con la salvedad de que, si redefinimos dentro del código HTML (entre las etiquetas **</style>**) estilos que habían quedado definidos en el archivo externo, los que se aplicarán serán los que hayamos redefinido.

Así, en el ejemplo anterior, aunque hubiésemos definido en **estilo.css** un color de fondo para la página, el color que prevalecería sería el definido a continuación de la importación: **#ffffcc**.

La diferencia entre este tipo de importación del tipo y la que hemos visto anteriormente:

```
<link rel="stylesheet" type="text/css" href="hoja.css">
```

Es que **@import url ("estilo.css")** se suele utilizar cuando hay unas pautas básicas en el trabajo con los estilos (que se definen en un archivo a importar) y unos estilos específicos para cada página, que se definen a continuación, dentro del código HTML entre las etiquetas **</style>**, como es el caso del ejemplo visto anteriormente.

Agregando Contenido a elementos

La propiedad **content** se emplea para generar nuevo contenido de forma dinámica e insertarlo en la página HTML. Como CSS es un lenguaje de hojas de estilos cuyo único propósito es controlar el aspecto o presentación de los contenidos, algunos diseñadores defienden que no es correcto generar nuevos contenidos mediante CSS.

El siguiente ejemplo muestra cómo añadir la palabra Capítulo delante del contenido de cada título de sección **<h1>**:

```
h1:before { content: "Capítulo "};
```

Los pseudo-elementos **:before** y **:after** se pueden utilizar sobre cualquier elemento de la página. El siguiente ejemplo añade la palabra Nota: delante de cada párrafo cuya clase CSS sea nota:

```
p.nota:before { content: "Nota: "};
```

Los contenidos insertados dinámicamente en un elemento son a todos los efectos parte de ese mismo elemento, por lo que heredan el valor de todas sus propiedades CSS.

Los dos valores más sencillos de la propiedad **content** son **none** y **normal**. En la práctica, estos dos valores tienen el mismo efecto ya que hacen que el pseudo-elemento no se genere.

```
#ultimo:after { content: " Fin de los 'contenidos' de la página."};
```

Las cadenas de texto sólo permiten incluir texto básico. Si se incluye alguna etiqueta HTML en la cadena de texto, el navegador muestra la etiqueta tal y como está escrita, ya que no las interpreta. Para incluir un salto de línea en el contenido generado, se utiliza el carácter especial `\A`

El siguiente valor aceptado por la propiedad `content` es una URL, que suele utilizarse para indicar la URL de una imagen que se quiere añadir de forma dinámica al contenido. La sintaxis es idéntica al resto de URL que se pueden indicar en otras propiedades CSS:

```
span.especial:after { content: url("imagenes/imagen.png");}
```

Otros valores que se pueden indicar en la propiedad `content` son **open-quote**, **close-quote**, **no-open-quote** y **no-close-quote**. Los dos primeros indican que se debe mostrar una comilla de apertura o de cierre respectivamente. Las comillas utilizadas se establecen mediante la propiedad **quotes**:

```
blockquote { quotes: "«" "»" '‘' '’' }  
blockquote:before { content: open-quote; }  
blockquote:after { content: close-quote; }
```

Los valores **no-open-quote** y **no-close-quote** se utilizan para no mostrar ninguna comilla en ese elemento, pero incrementando el nivel de anidamiento de las comillas. De esta forma se puede evitar mostrar una comilla en un determinado elemento mientras se mantiene la jerarquía de comillas establecida por la propiedad **quotes**.

Uno de los valores más avanzados de la propiedad content es **attr()**, que permite obtener el valor de un atributo del elemento sobre el que se utiliza la propiedad **content**. En el siguiente ejemplo, se modifican los elementos **<abbr>** y **<acronym>** para que muestren entre paréntesis el valor de sus atributos **title**:

```
abbr:after, acronym:after { content: " (" attr(title) ")"};
```

El valor de la propiedad content anterior en realidad es la combinación de tres valores:

- | Cadena de texto " (", que es el paréntesis de apertura tras el cual se muestra el valor del atributo title.
- | Atributo title del elemento obtenido mediante attr(title)
- | Cadena de texto ")", que es el paréntesis de cierre que se muestra detrás del valor del atributo title.

Si el elemento no dispone del atributo solicitado, la función **attr(nombre_del_atributo)** devuelve una cadena de texto vacía. Utilizando attr() solamente se puede obtener el valor de los atributos del elemento al que se aplica la propiedad **content**. La función **attr()** es muy útil por ejemplo para mostrar la dirección a la que apuntan los enlaces de la página:

```
a:after { content: " (" attr(href) ")"; }
```

Los últimos valores que se pueden indicar en la propiedad content son los contadores creados con las propiedades **counter-increment** y **counter-reset**. Los contadores más sencillos se muestran con la función **counter(nombre_del_contador)**. El siguiente ejemplo crea dos contadores llamado capítulo y sección para utilizarlos con los elementos **<h1>** y **<h2>**:

```
body { counter-reset: capítulo; }
h1 { counter-reset: seccion; }
h1:before { content: "Capítulo " counter(capítulo) ". ";
  counter-increment: capítulo;}
h2:before { content: counter(capítulo) "." counter(seccion) ";
  counter-increment: seccion; }
```

En el ejemplo anterior, se crea e inicializa su valor a 0 un contador llamado capítulo cuando se encuentre el elemento **<body>**, es decir, al comienzo de la página. Además, se crea e inicializa su valor a 0 otro contador llamado sección cada vez que se encuentra un elemento **<h1>** en la página.

Posteriormente, se añade de forma dinámica a los elementos **<h1>** y **<h2>** el contenido generado mediante los contadores. Los elementos **<h1>** utilizan el contador capítulo y lo incrementan en una unidad cada vez que lo utilizan. Los elementos **<h2>** utilizan los dos contadores para generar un contenido que muestra su numeración completa. Además, los elementos **<h2>** actualizan el valor del contador sección.

Cuando un mismo elemento inicializa/actualiza un contador y lo utiliza en la propiedad **content**, en primer lugar se inicializa/actualiza y después, el valor actualizado es el que se utiliza mediante **counter()**.

Además de mostrar el valor de un contador básico, la función counter() permite indicar el estilo con el que se muestra el valor del contador. La lista de estilos permitidos son los mismos que los de la propiedad **list-style-type**.

El siguiente ejemplo modifica el anterior para mostrar el valor de los contadores en números romanos:

```
body { counter-reset: capitulo;}
h1 { counter-reset: seccion;}
h1:before { content: "Capítulo " counter(capitulo, upper-roman) ". ";
  counter-increment: capitulo;}
h2:before {
  content: counter(capitulo, upper-roman) "." counter(seccion, upper-roman) " ";
  counter-increment: seccion;}
```

Los estilos de los contadores también se pueden emplear para no mostrar el valor de los contadores en algunos elementos:

```
p { counter-increment: parrafos;}
p:before { content: counter(parrafos);}
#especial p:before { content: counter(parrafos, none);}
```

Aunque el valor de los contadores siempre es numérico, también se pueden emplear estilos gráficos como **square**, **disc** o **circle**:

```
h2 { counter-increment: seccion;}
h2:before { content: counter(seccion, disc);}
```

La función **counter()** solamente muestra el valor de un contador. Por su parte, la función **counters()** se utiliza para mostrar de forma simultánea el valor de todos los contadores asociados con el elemento. Como se explica en la descripción de las propiedades **counter-increment** y **counter-reset**, los contadores se pueden anidar y un mismo elemento puede tener asociados varios contadores diferentes con el mismo nombre.

El siguiente ejemplo muestra unas reglas CSS que crean un contador para los elementos de una lista :

```
ol { counter-reset: elemento; list-style-type: none;}
li:before { content: counter(elemento) ". "; counter-increment:
elemento;}
```

Si se considera el siguiente código HTML:

```
<ol>
  <li>Elemento</li>
  <li>Elemento</li>
  <li>Elemento
    <ol>
      <li>Elemento</li>
      <li>Elemento
        <ol>
          <li>Elemento</li>
          <li>Elemento</li>
          <li>Elemento</li>
```



```
</ol>
</li>
<li>Elemento</li>
</ol>
</li>
<li>Elemento</li>
</ol>
```

Si se aplican las reglas CSS al código HTML anterior, se crean tres contadores diferentes con el mismo nombre (elemento) . Sin embargo, si se utiliza la función **counters()** en las reglas CSS anteriores:

```
ol { counter-reset: elemento; list-style-type: none;}
li:before { content: counters(elemento, '. ') ". ";
counter-increment: elemento; }
```

En el ejemplo anterior, cada vez que se encuentra un elemento **** se crea un contador llamado elemento. Por este motivo, los elementos **** anidados se ven afectados por varios contadores llamados elemento. La función **counter()** sólo muestra el valor del contador que afecta más directamente al elemento, mientras que la función **counters()** muestra todos los contadores empezando desde el más externo hasta llegar al más interno.

El segundo argumento de la función **counters()** es una cadena de texto que se emplea para separar los valores de los diferentes contadores.

Por último, la función **counters()** también permite indicar el estilo con el que se muestra el valor de los contadores. De esta forma, el siguiente ejemplo modifica el anterior para mostrar el valor de todos los contadores en números romanos:

```
ol { counter-reset: elemento; list-style-type: none;}  
li:before { content: counters(elemento, ' ', upper-roman) ". ";  
            counter-increment: elemento;}
```

Sombras

Una de las propiedades interesantes que nos ofrece CSS3 es la capacidad de crear y configurar el efecto de sombra. Las sombras se pueden aplicar tanto a bloques, como a texto. Entre otras cosas, podremos elegir el **color**, **tamaño**, **desenfoque** y **desplazamiento** de la sombra (**drop shadow**).

La sombra paralela más simple y más frecuente probablemente sea una sombra gris que se extiende algunos píxeles a partir del cuadro, con algo de desenfoque.

| **box-shadow:** nos permite crear un sombreado todos los elementos HTML (elementos soportados como bloques “Divs”).

| **text-shadow:** nos permite crearle sombras a los textos a los que apliquemos esta propiedad.

Por ejemplo si agregamos una sombra con desplazamiento vertical y horizontal de solo cinco píxeles, además de un ligero desenfoque de 5 píxeles sin difusión de color gris claro codificaremos la siguiente línea:

```
box-shadow: 5px 5px 5px lightgray;
```

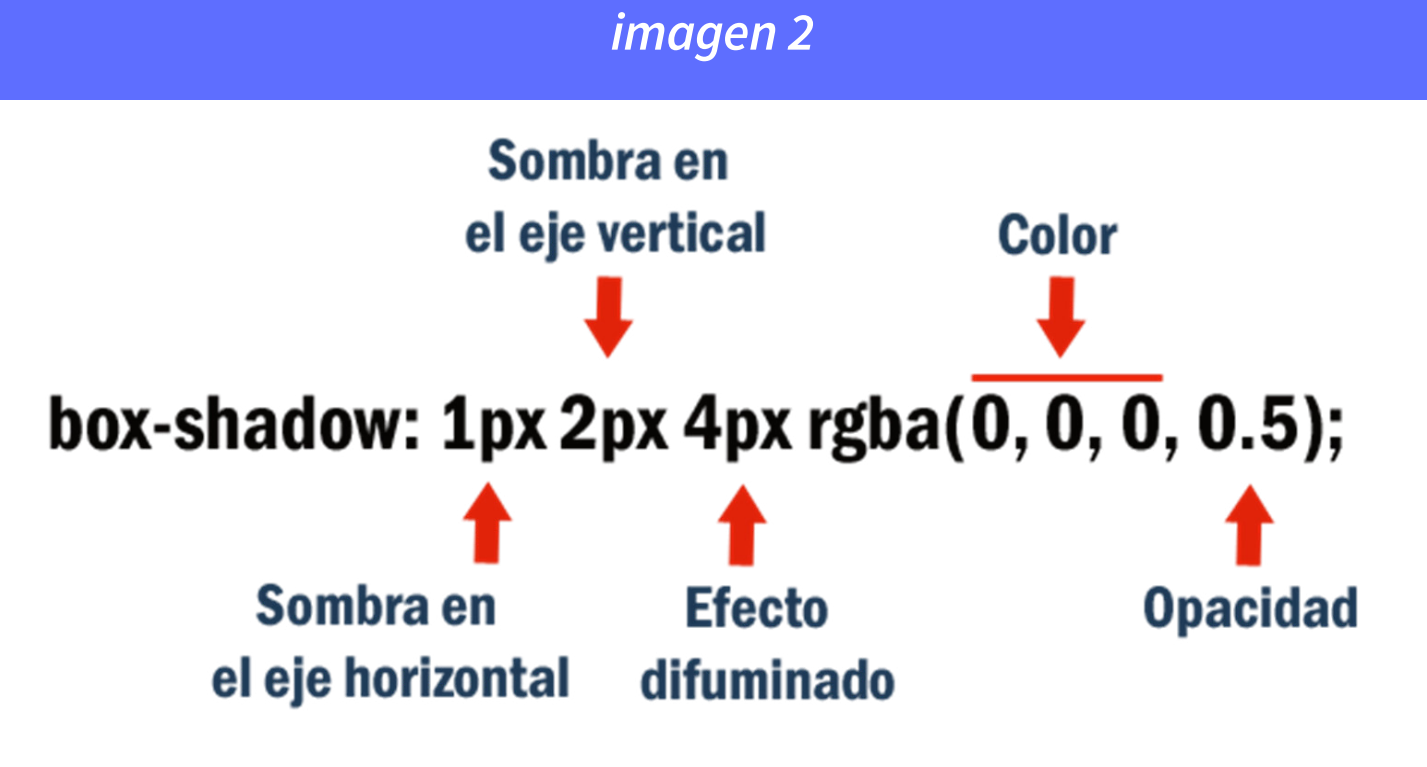
Donde tendremos la siguiente sintaxis: imagen 2

Por orden de aparición, los valores que se indican en box-shadow son:

| **Desplazamiento horizontal de la sombra:** La sombra de un elemento suele estar un poco desplazada con respecto al elemento que la produce y su posición será en función del ángulo con el que llegue la luz. Si la sombra quisiéramos que apareciera un poco hacia la izquierda del elemento original que la produce, pondríamos un valor negativo a este atributo. Cuanto más desplazamiento tenga una sombra, el elemento que la produce parecerá que está más separado del lienzo de la página.

| **Desplazamiento vertical de la sombra:** El segundo valor que colocamos en el atributo **box-shadow** es el desplazamiento vertical de la sombra con respecto a la posición del elemento que la produce. Este valor es similar al desplazamiento horizontal. Valores positivos indican que la sombra aparecerá hacia abajo del elemento y valores negativos harán que la sombra aparezca desplazada un poco hacia arriba.

| **Difuminado:** El tercer valor indica cuánto queremos que esté difuminado el borde de la sombra. Si el difuminado fuera cero, querría decir que la sombra no tiene ningún difuminado y aparece totalmente definida.



| **Color de la sombra:** El último atributo que se indica en el atributo **box-shadow** es el color de la sombra. Generalmente las sombras en el mundo real tienen un color negro o grisáceo, pero con CSS3 podremos indicar cualquier gama de color para hacer la sombra, lo que nos dará bastante más versatilidad a los diseños gracias a la posible utilización de sombras en distintos colores, que puedan combinar mejor con nuestra paleta.

Imagen 3

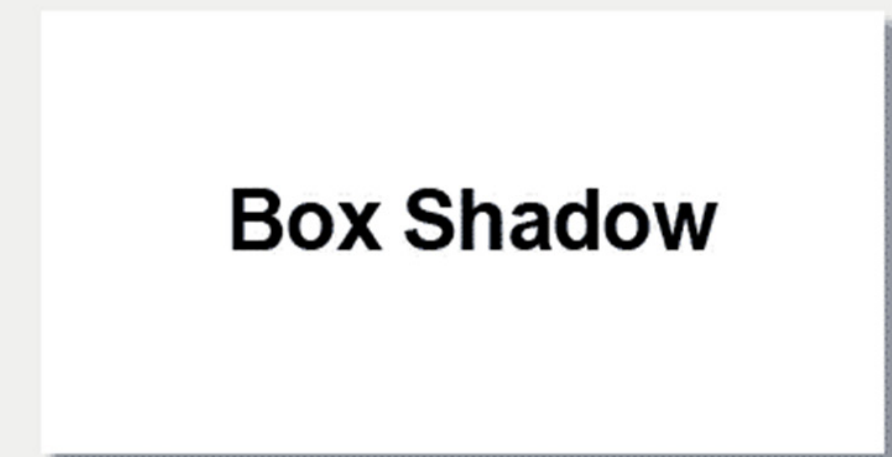
```
#caja1{-Webkit-box-shadow: 5px 5px 5px # 888;  
-Moz-box-shadow: 5px 5px 5px # 888;  
box-shadow: 5px 5px 5px # 888; }
```

Este otro ejemplo es para una sombra un poco menor, también desplazada hacia abajo y a la derecha y con un difuminado de 2 píxeles. Además hemos indicado un color amarillo claro para la sombra, por lo que, para verla bien, tendríamos que colocar este elemento sobre un fondo oscuro.

```
#sombraredondeada{ background-color: #090;  
color: #fff; width: 400px; padding: 10px;  
-moz-border-radius: 7px; -webkit-border-radius: 7px;  
box-shadow: 15px -10px 3px #000;  
-webkit-box-shadow: 15px -10px 3px #000;  
-moz-box-shadow: 15px -10px 3px #000;}
```

imagen 3

creando efecto de Sombra a div caja1



Desplazamiento Horizontal eje X Desplazamiento Vertical eje Y Desenfoque Radial Color

```
box-shadow: 5px 5px 5px #888;  
-webkit-box-shadow: 5px 5px 5px #888;  
-moz-box-shadow: 5px 5px 5px #888;
```

Texto con sombra

CSS tiene una propiedad que permite añadir sombra al texto. Lleva cuatro argumentos: el color de la sombra, el desplazamiento horizontal (donde positivo significa hacia la derecha), el desplazamiento vertical (donde positivo significa hacia abajo) y el desenfoque (0 significa una sombra nítida). Por ejemplo:

```
h3 { text-shadow: red 0.2em 0.3em 0.2em }  
.elemento { box-shadow: 2px 2px 5px #999;}
```

Imagen 4

| La primera medida es obligatoria e indica el **desplazamiento horizontal** de la sombra. Si el valor es positivo, la sombra se desplaza hacia la derecha y si es negativo, se desplaza hacia la izquierda.

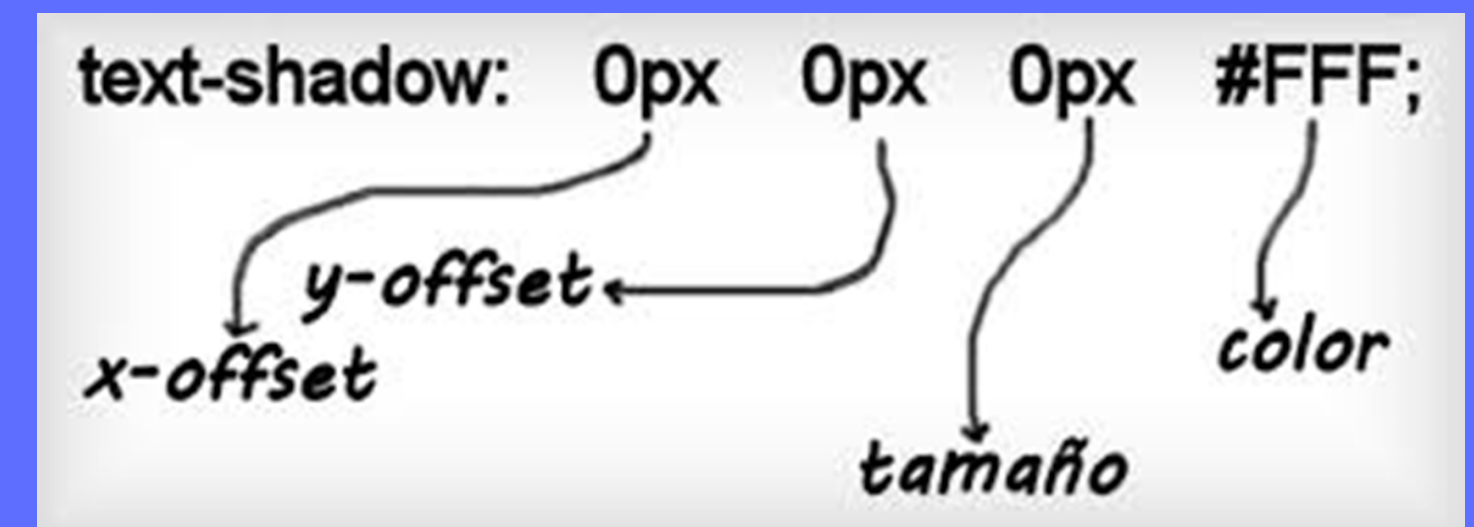
| La segunda medida también es obligatoria e indica el **desplazamiento vertical** de la sombra. Si el valor es positivo, la sombra se desplaza hacia abajo y si es negativo, se desplaza hacia arriba.

| La tercera medida es opcional e indica el radio utilizado para **difuminar** la sombra. Cuanto más grande sea su valor, más borrosa aparece la sombra. Si se utiliza el valor 0, la sombra se muestra como un color sólido.

| La cuarta medida también es opcional e indica el radio con el que se **expande** la sombra. Si se establece un valor positivo, la sombra se expande en todas direcciones. Si se utiliza un valor negativo, la sombra se comprime.

| El color indicado es directamente el color de la sombra que se muestra

imagen 4



Las opciones del filtro **shadow** son mucho más limitadas que las de la propiedad **box-shadow**. Su sintaxis es la habitual de los filtros de Internet Explorer y las opciones son:

| **color**, establecido mediante el formato hexadecimal (ejemplo: #CC0000).

| **direction**, dirección hacia la que se desplaza la sombra. Su valor se indica en grados y sólo se permiten los valores 0, 45, 90, 135, 180, 225, 270 y 315.

| **strength**, distancia en píxeles hasta la que se extiende la sombra.

A continuación se modifica la regla CSS anterior para incluir el filtro de Internet Explorer que muestra una sombra similar:

```
.elemento {  -webkit-box-shadow: 2px 2px 5px #999;  
             -moz-box-shadow: 2px 2px 5px #999;  
             filter: shadow(color=#999999, direction=135, strength=2);}
```

Texto con sombra borrosa

La forma más sencilla de la propiedad "**text-shadow**" tiene dos partes: un **color** y un **desplazamiento**. El resultado es una sombra nítida con el desplazamiento indicado. Pero también puede hacerse que el desplazamiento sea impreciso, lo que da por resultado una sombra más o menos borrosa. La cantidad de imprecisión se da como otro desplazamiento. Aquí se muestran dos líneas, una con un poco de imprecisión (0.05em) y la otra con mucha imprecisión (0.2em):

```
h3.a {text-shadow: 0.1em 0.1em 0.05em #333}  
h3.b {text-shadow: 0.1em 0.1em 0.2em black}
```

Texto blanco legible

El uso de sombras puede hacer más legible el texto cuanto hay poco contraste entre el primer plano y el fondo. Lo que sigue es un ejemplo de texto blanco contra fondo azul pálido, primero sin sombra y luego con ella:

```
h3 {color: white}  
h3.a {color: white; text-shadow: black 0.1em 0.1em 0.2em}
```

Varias sombras

También es posible poner más de una sombra. En general, el resultado es un tanto extraño:

```
h3 {text-shadow: 0.2em 0.5em 0.1em #600,  
               -0.3em 0.1em 0.1em #060, 0.4em -0.3em 0.1em #006}
```

Pero colocando acertadamente dos sombras, una oscura y la otra clara, el efecto puede ser útil:

```
h3.a {text-shadow: -1px -1px white, 1px 1px #333}  
h3.b {text-shadow: 1px 1px white, -1px -1px #333}
```

Esto es un poco peligroso, como podrá apreciar si su navegador no admite la propiedad "**text-shadow**". De hecho, en este ejemplo los colores del fondo y del texto son casi iguales (#CCCCCC y #D1D1D1), de modo que sin las sombras apenas existe algún contraste.

Trazar letras como contornos

El ejemplo con dos sombras de la versión previa se puede llevar todavía más allá. Con cuatro sombras, es posible dar a las letras un contorno:

```
h3 {text-shadow: -1px 0 black, 0 1px black, 1px 0 black, 0 -1px black}
```

imagen 5

imagen 5

Texto contorneado

Texto contorneado

Brillo de neón

Si se pone una sombra borrosa justo detrás del texto, es decir, con desplazamiento igual a cero, el efecto es un resplandor alrededor de las letras. Si el resplandor de una sola sombra no es suficientemente intenso, basta repetir la misma sombra unas pocas veces:

```
h3.a {text-shadow: 0 0 0.2em #8F7}  
h3.b {text-shadow: 0 0 0.2em #F87, 0 0 0.2em #F87}  
h3.c {text-shadow: 0 0 0.2em #87F, 0 0 0.2em #87F, 0 0 0.2em #87F}
```

Efecto Pixelart

Con un poco más de imaginación podemos conseguir efectos de lo más diverso. En este caso hemos hecho una prueba que da un resultado de diseño "pixelart", de aquellos gráficos creados píxel a píxel de los juegos de antaño.

```
h1.pixelart{  
  text-shadow: 1px 1px #666, 2px 2px #86D6D3, 3px 3px #666, 4px 4px  
  #86D6D3;   color: #ccc;}
```

imagen 6

imagen 6

Este texto tiene un pixelart

Efecto de fuego

Si usamos varias sombras de colores anaranjados podemos conseguir un efecto de fuego. Nos toca hacer un poco de prueba y ensayo para conseguir un resultado realista, pero se puede conseguir algo interesante.

```
h2.fuego{
  text-shadow: 0 0 20px #fefcc9, 2px -2px 3px #feec85, -4px -4px 5px
#ffae34, 5px -10px 6px #ec760c, -5px -12px 8px #cd4606, 0 -15px 20px
#973716, 2px -15px 20px #451b0e; color: #666;}
```

Degradado

Los degradados implementan un gradiente de color, que pasa de un estado a otro a lo largo del fondo de los elementos HTML, ya sea capas, elementos de listas, botones, etc. Dichos degradados se obtendrán por medio de la especificación de una serie de características, como la posición inicial, la dirección hacia donde se realizará, si es circular o linear, y los colores que se incorporarán en cada uno de los pasos del gradiente. Podemos definirlos un degradado así:

```
background-image: linear-gradient(izquierda arriba , de negro, a
blanco , la mitad)
```

Degradado lineal

Son aquellos en los que se crea un degradado que va de un color a otro de manera lineal. Puede ser de arriba a abajo, de izquierda a derecha y viceversa. Incluso se puede conseguir un degradado lineal con cualquier ángulo.

```
div{  height: 130px;  width: 630px;
  background: -webkit-linear-gradient(orange, pink);
  background: -moz-linear-gradient(orange, pink);
  background: -o-linear-gradient(orange, pink); }
```

Imagen 7

Degradado circular

En ellos se implementa un degradado que se distribuye radialmente, desde un punto del elemento hacia fuera, de manera circular, que puede tener el mismo valor de radio (para hacer degradados en círculos perfectos) o con valores de radio variables (lo que generaría elipses).

El valor que asignamos a **background** en este caso será por medio del atributo "radial-gradient", además de toda la serie de parámetros necesarios para definir el degradado según nuestras intenciones.

imagen 8

```
div.circular{
  background: -webkit-radial-gradient(#0f0, #06f);
  background: -moz-radial-gradient(#0f0, #06f);
  border: 1px solid #333;  height: 200px;  width: 250px; }
```

imagen 7

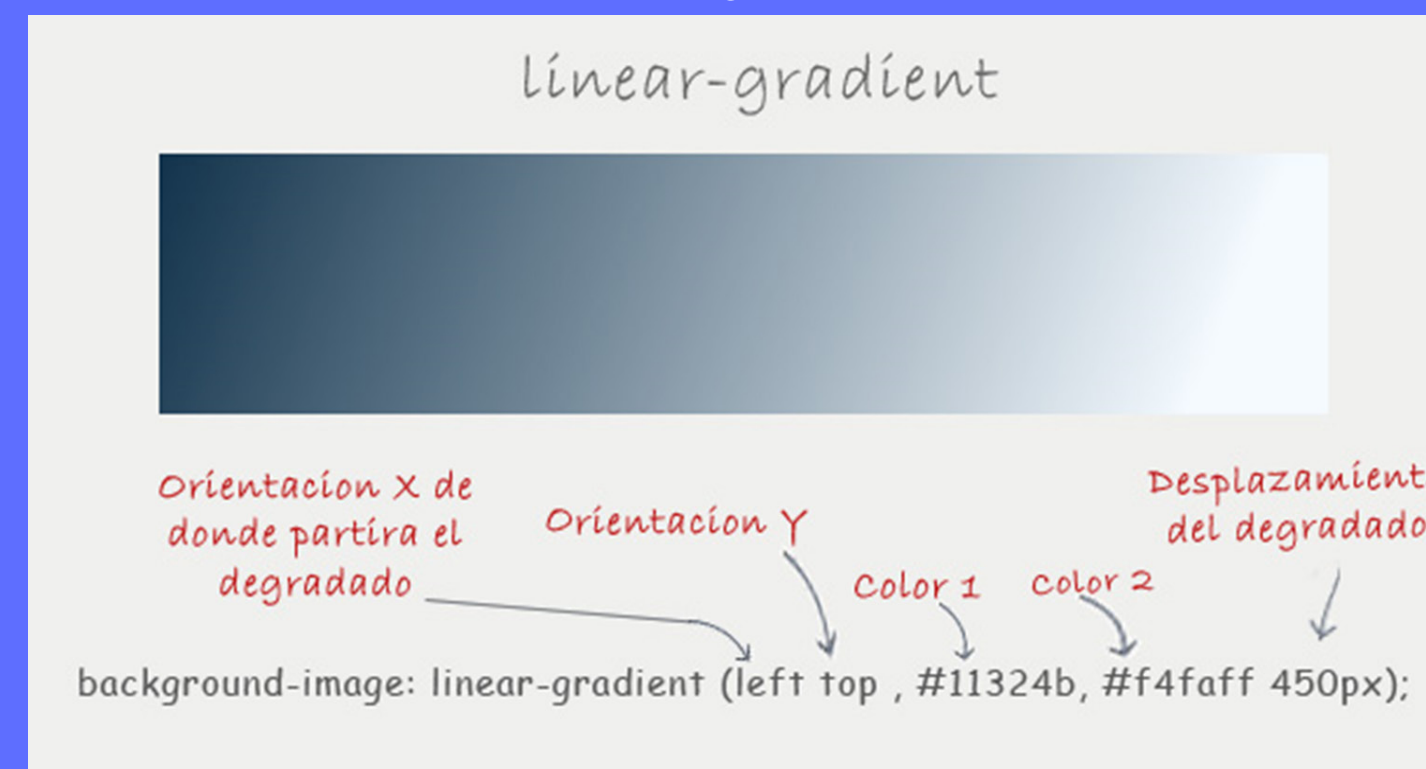
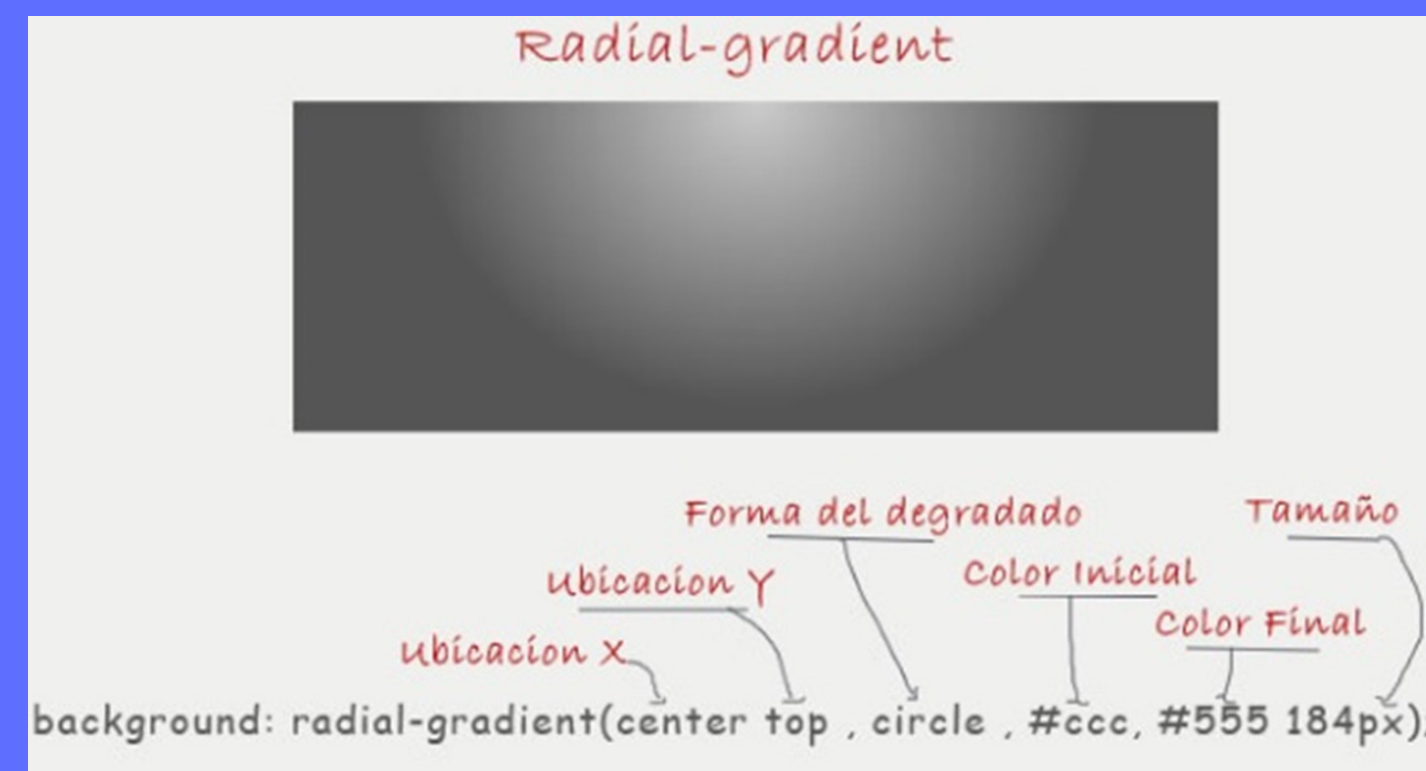


imagen 8



Para asegurarse que un degradado funciona en la mayoría de las plataformas, de momento estamos obligados a escribir las reglas de estilos con etiquetas propietarias para cada navegador. Osea, para definir un degradado deberíamos escribir todos estos estilos:

```
background: -webkit-linear-gradient(orange, pink);
background: -moz-linear-gradient(orange, pink);
background: -o-linear-gradient(orange, pink);
background: linear-gradient(orange, pink);
```

Transparencia

El valor de la propiedad **opacity** se establece mediante un número decimal comprendido entre 0.0 y 1.0. La interpretación del valor numérico es tal que el valor **0.0** es la **máxima transparencia** (el elemento es invisible) y el valor **1.0** se corresponde con la **máxima opacidad** (el elemento es completamente visible). De esta forma, el valor **0.5** corresponde a un elemento **semitransparente** y así sucesivamente.

En el siguiente ejemplo, se establece la propiedad **opacity** con un valor de **0.5** para conseguir una **transparencia** del **50%** sobre dos de los elementos **<div>**:

```
#segundo, #tercero { opacity: 0.5;}
#primero { background-color: blue;}
#segundo { background-color: red;}
#tercero { background-color: green;}
```

Los **filtros** permiten aplicar operaciones complejas a los elementos de la página. Los filtros de Internet Explorer se dividen en **estáticos** y de **transición**. Los filtros estáticos se utilizan básicamente para crear efectos gráficos sobre los elementos, normalmente imágenes. Entre los filtros estáticos se encuentra el filtro **alpha**, que permite controlar la opacidad de un elemento de la página. La sintaxis completa del filtro alpha es muy compleja porque sus posibilidades son numerosas. No obstante, la sintaxis necesaria para establecer solamente el nivel de transparencia de un elemento es muy sencilla:

```
#segundo, #tercero { filter: alpha(opacity=50);}
```

El valor de la opción **opacity** del filtro **alpha** se establece mediante un número entero comprendido entre 0 (el elemento es invisible) y 100 (el elemento es completamente opaco). El valor 50 del ejemplo anterior hace que el elemento sea semitransparente. A continuación se muestra la solución compatible con todos los navegadores para que un elemento de la página sea semitransparente:

```
selector { opacity: 0.5; filter: alpha(opacity=50);}
```

Transición

Una transición es un **cambio progresivo de un elemento**. Aplicado a los estilos CSS significa que dentro de una propiedad, para pasar de un valor a otro, esto se puede hacer de manera progresiva, pasando por los valores intermedios. Esto la mayoría de las veces provoca un efecto de animación, sobre todo en propiedades que tienen que ver con el tamaño, la posición o el color. Las transiciones sólo se dan en las propiedades CSS cuyo valor tenga un componente numérico (número, medidas, colores, tiempo).

Las transiciones en CSS3 son creadas por una serie de propiedades que controlarán los elementos, las propiedades CSS y el tiempo entre otras cosas. También podemos hacer transiciones de múltiples propiedades, para ello indicamos cada transición separada por coma:

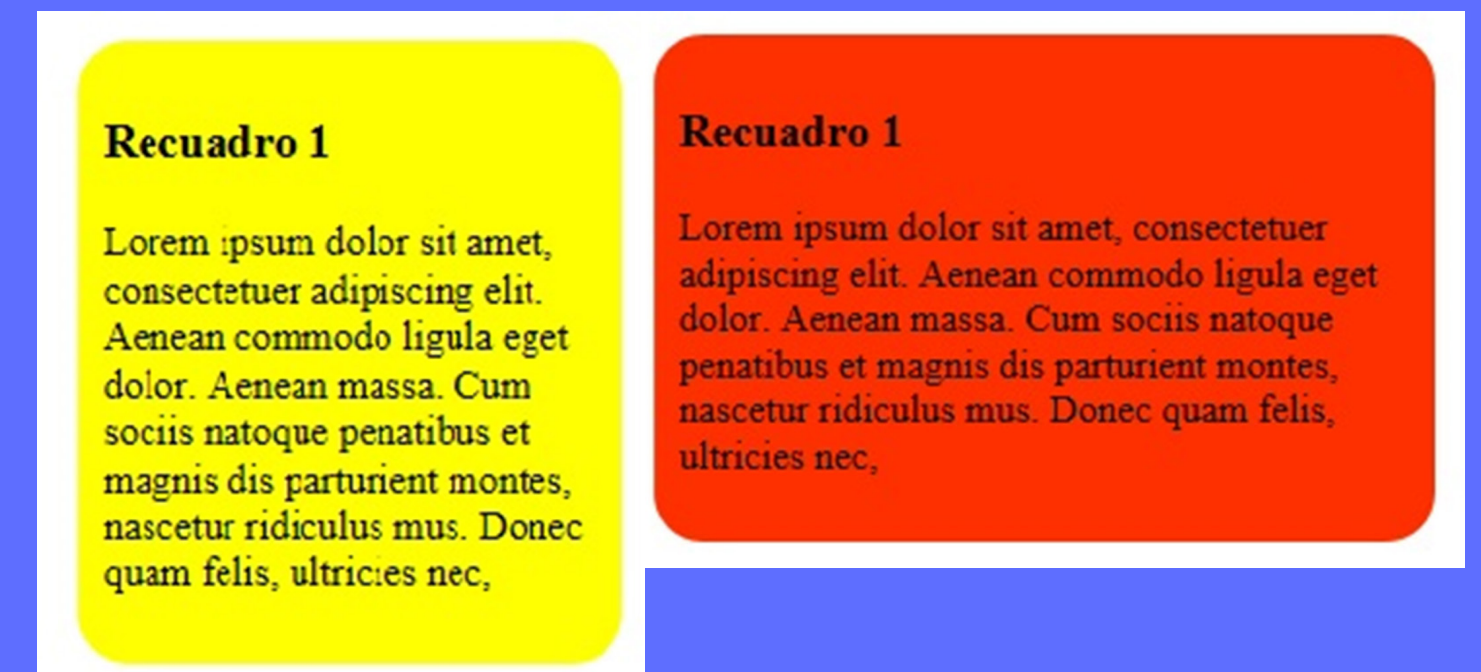
```
Elemento {  
  transition: [nombre de propiedad] [duración de la transición],  
             [nombre de propiedad] [duración de la transición],  
             [nombre de propiedad] [duración de la transición] ; }
```

Por ejemplo si queremos que el recuadro modifique su ancho y cambie de color luego debemos codificar lo siguiente:

```
#recuadro1{ border-radius: 20px;    background-color:#ff0;  
width:200px; padding:10px;  
transition:width 1s,  
    background-color 8s;  
-moz-transition:width 1s,  
    background-color 8s;  
-ms-transition:width 1s,  
    background-color 8s;  
-webkit-transition:width 1s,  
    background-color 8s;  
-o-transition:width 1s,  
    background-color 8s; }  
#recuadro1:hover{ width:300px; background-color: #f00;}
```

Como vemos en la propiedad transition indicamos la propiedad **width** con una duración de un segundo y la propiedad **background-color** con un valor de 8 segundos. Esto significa que cuando dispongamos la flecha del mouse dentro del div se lanzarán ambas transiciones que tienen duraciones distintas (cambiará de 200 píxeles a 300 píxeles en el lapso de un segundo y también cambiará del color amarillo al rojo en forma gradual en un lapso de 8 segundos) *Imagen 9*

imagen 9



Ejemplo completo:

```
<!DOCTYPE html>
<html>
<head> <title>Prueba de transiciones</title>
<style type="text/css">
#recuadro1{ border-radius: 20px;   background-color:#ff0;
width:200px; padding:10px;
transition:width 1s,
    background-color 8s;
-moz-transition:width 1s,
    background-color 8s;
-ms-transition:width 1s,
    background-color 8s;
-webkit-transition:width 1s,
    background-color 8s;
-o-transition:width 1s,
    background-color 8s;}
#recuadro1:hover{ width:300px; background-color: #f00;}
body { background:white; margin:50px;}
</style> </head>
<body>
<div id="recuadro1">
<h3>Recuadro 1</h3>
```

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean  
commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus  
et magnis dis parturient montes, nascetur ridiculus mus. Donec quam  
felis, ultricies nec, </p>  
</div>  
</body>  
</html>
```

Las transiciones de CSS3 pueden tomar estos valores:

- | transition-property
- | transition-duration
- | transition-delay
- | transition-timing-function

Transformaciones

De las transformaciones de CSS3 en 2D, las más usadas son:

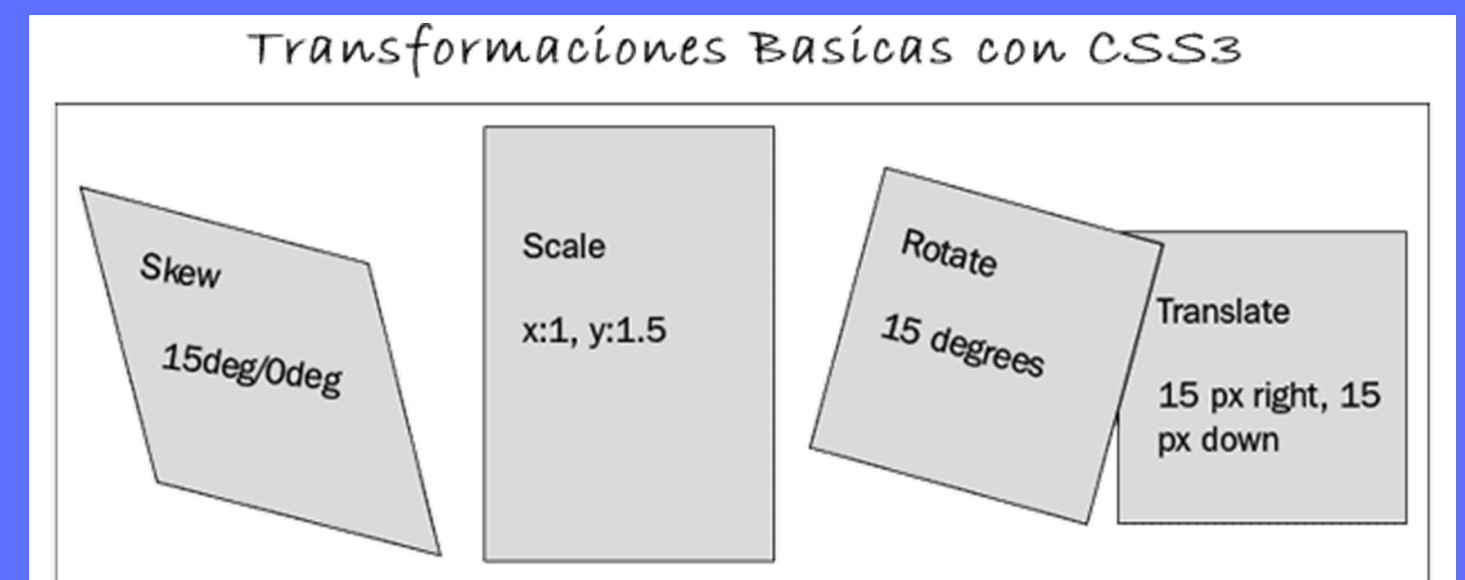
| **Rotate.** nos permite rotar un elemento dándole un ángulo de giro en grados.

| **Scale.** nos permite escalar un elemento, toma valores positivos y negativos y se le pueden poner decimales.

| **Translate.** nos permite trasladar un elemento a la vez en el eje de las X y de las Y, dándole las coordenadas iniciales y finales.

imagen 10

imagen 10



A continuación detallaremos cada una de ellas:

Rotar

Para la rotación de un objeto se puede utilizar **trans-form:rotate(grados)**, también nos permite rotar solo un eje cuando se trata de un objeto 3D utilizando **rotateX(grados)**, **rotateY(grados)** o **rotateZ(grados)**

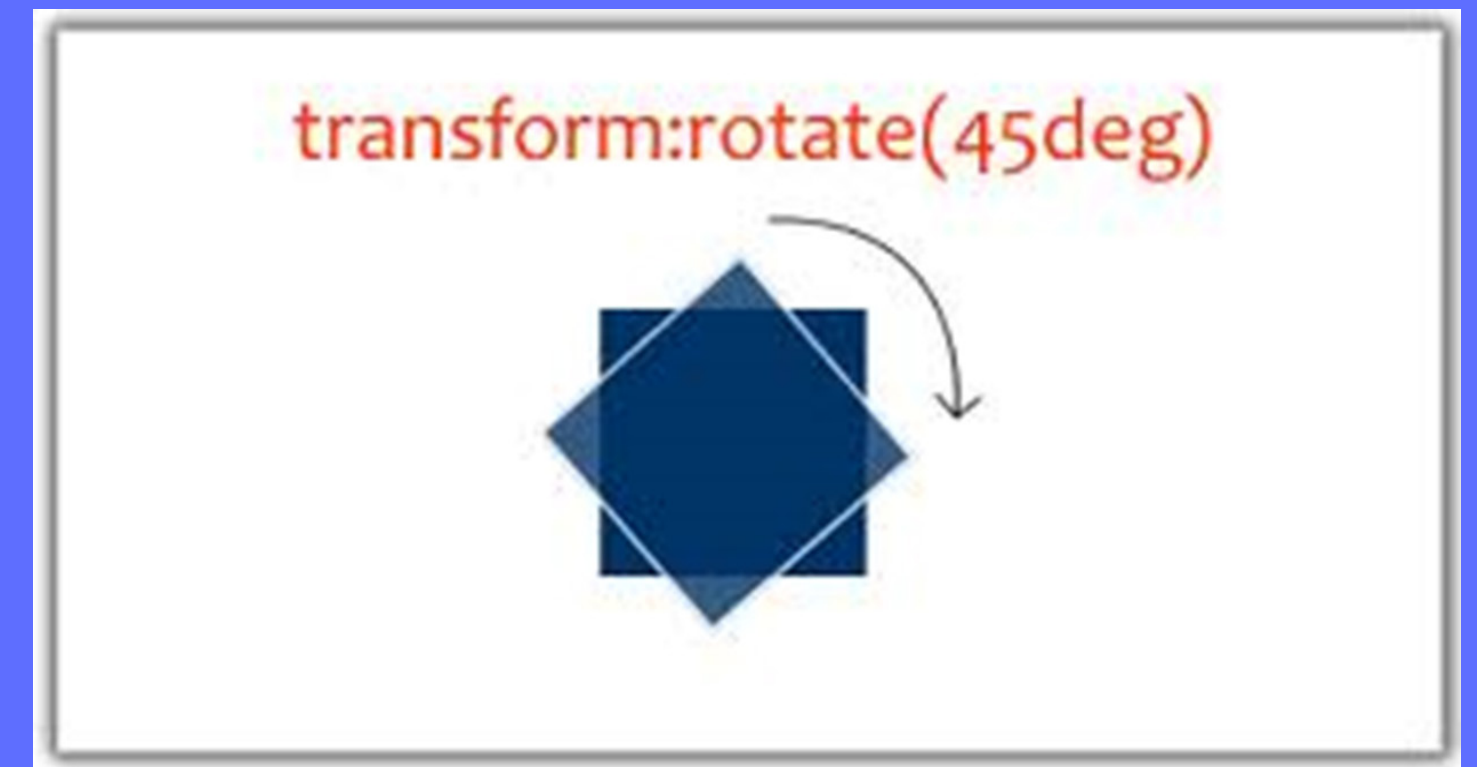
```
#rotacion{ -webkit-transition: all 1s ease-in-out;
-moz-transition: all 1s ease-in-out;
-o-transition: all 1s ease-in-out;
transition: all 1s ease-in-out; }
#rotacion:hover{-webkit-transform: rotate(360deg);
-moz-transform: rotate(360deg);
-o-transform: rotate(360deg);
-ms-transform: rotate(360deg);
transform: rotate(360deg); }
```

imagen 11

```
.ejemplo { -webkit-transform: rotate (45deg);
-moz-transform: rotate (45deg);
-o-transform: rotate (45deg);
-ms-transform: rotate (45deg);
transform: rotate (45deg); }
```

Fíjate que los grados se marcan en positivo si el elemento se rota en el sentido de las agujas del reloj y en negativo si es al revés.

imagen 11



Sesgar

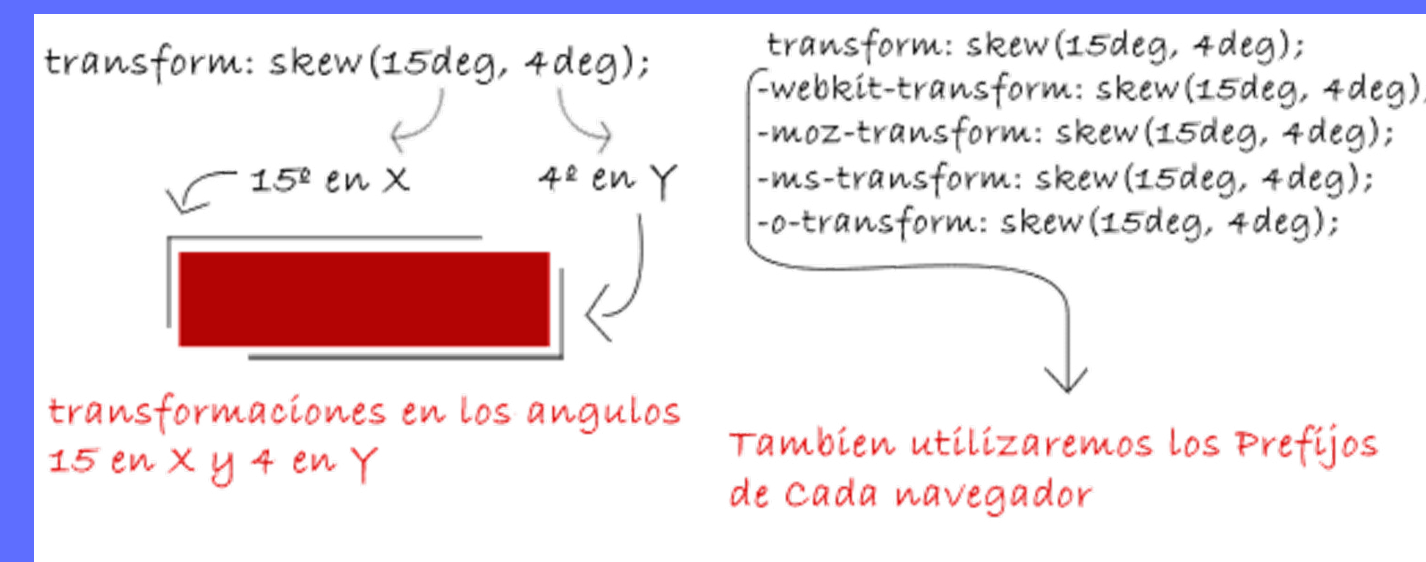
Esta función nos va a permitir inclinar o aplicar un tipo de perspectiva a un elemento en el documento HTML, como los demás tenemos que especificar las coordenadas de sus angulos para su inclinación, (X, Y) el primero **X** que se aplica **Horizontalmente** y **Y** que se aplica **verticalmente**.

Para sesgar un objeto debes utilizar transform: **skew**(angulo x, angulo y) o al igual que las anteriores también permite sesgar solo un eje utilizando **skewX**(angulo) o **skewY**(angulo)

```
#sesgo{-webkit-transition: all 1s ease-in-out;
-moz-transition: all 1s ease-in-out;
-o-transition: all 1s ease-in-out;
transition: all 1s ease-in-out;}
#sesgo:hover{-webkit-transform: skew(25deg,180deg);
-moz-transform: skew(25deg,180deg);
-o-transform: skew(25deg,180deg);
-ms-transform: skew(25deg,180deg);
transform: skew(25deg,180deg); }
```

Imagen 12

imagen 12



```
div { background: #ccc; box-shadow: 2px 2px 3px #777;
margin: 0 auto; transform: skew(15deg, 4deg);
-webkit-transform: skew(15deg, 4deg);
-moz-transform: skew(15deg, 4deg);
-ms-transform: skew(15deg, 4deg);
-o-transform: skew(15deg, 4deg); width: 350px; }
```

imagen 13

Desplazamiento

Para mover un elemento es necesario utilizar **transform: translate(desplazamiento-x, desplazamiento-y)** o si solo quieres desplazar el objeto por un eje puedes trabajar con **translateX()** o **translateY()**

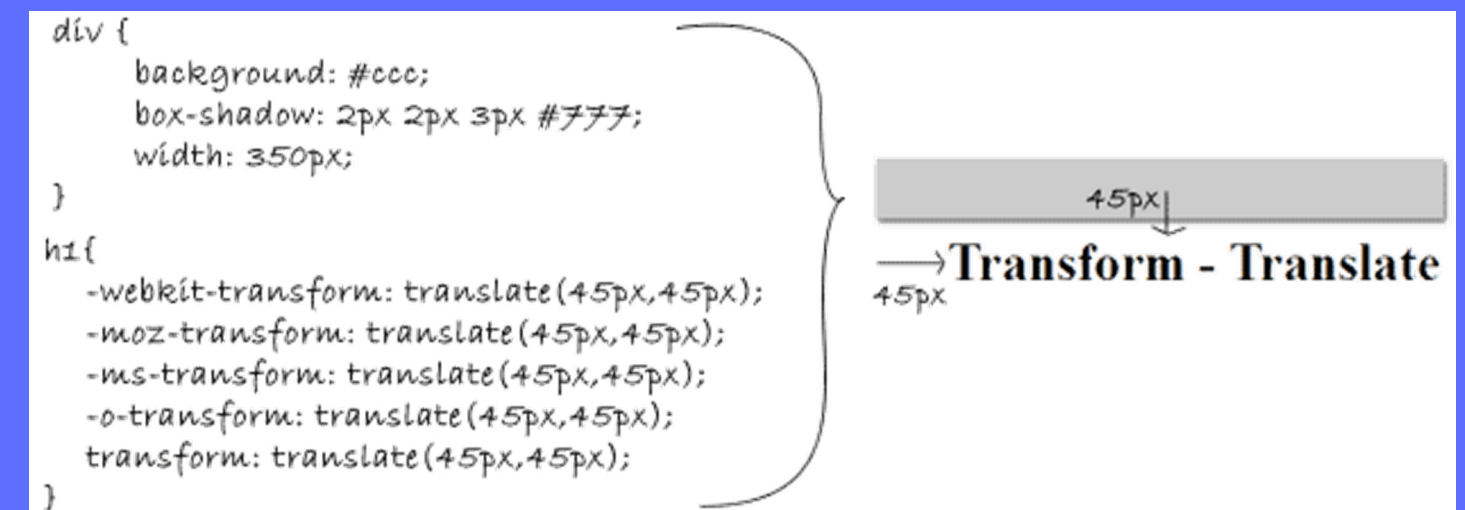
```
#desplazar{-webkit-transition: all 1s ease-in-out;
-moz-transition: all 1s ease-in-out;
-o-transition: all 1s ease-in-out;
transition: all 1s ease-in-out;}
#desplazar:hover{-webkit-transform: translate(250px, 20px);
-moz-transform: translate(250px, 20px);
-o-transform: translate(250px, 20px);
-ms-transform: translate(250px, 20px);
transform: translate(250px, 20px); }
```

imagen 14

imagen 13



imagen 14



Escalar

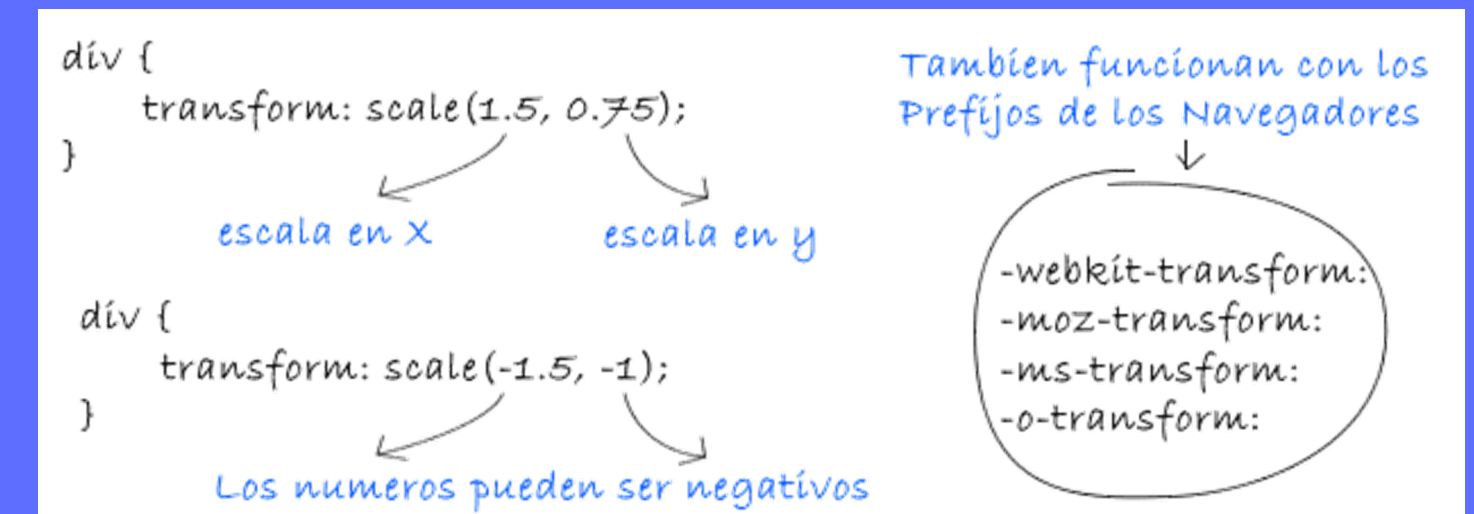
Otra función de la propiedad transform es **scale**, esta función nos va a permitir escalar un elemento tanto horizontal como verticalmente entorno a su origen y lo hacemos por medio de su (x,y).

```
h1{ transform: scale(1.9, 1);  
  -webkit-transform: scale(1.9, 1);  
  -moz-transform: scale(1.9, 1);  
  -ms-transform: scale(1.9, 1);  
  -o-transform: scale(1.9, 1); }
```

Si deseas aplicar varias transformaciones a la vez solo tienes que encadenar las transformaciones, el orden en el que las apliques las transformaciones puede hacer que el efecto cambie, veremos un ejemplo a continuación:

```
#transformaciones{ -webkit-transition: all 1s ease-in-out;  
  -moz-transition: all 1s ease-in-out;  
  -o-transition: all 1s ease-in-out;  
  transition: all 1s ease-in-out; }  
#transformaciones:hover{  
  -webkit-transform: translateX(200px) rotate(360deg) scale(.7) ;  
  -moz-transform: translateX(200px) rotate(360deg) scale(.7);  
  -o-transform: translateX(200px) rotate(360deg) scale(.7);  
  -ms-transform: translateX(200px) rotate(360deg) scale(.7);  
  transform: translateX(200px) rotate(360deg) scale(.7);  
}
```

imagen 15



Ejemplos y Ejercicios comentados

De ahora en más expondremos ejercicios resueltos, comentados y detallados en forma completa aplicando todo lo visto de **CSS3** y **HTML5** hasta el momento, como siempre te aconsejo que practiques y codifiques los ejemplos ya que esto es un buen método para aprender el código.

Listas con CSS

En este ejercicio utilizaremos una lista, **pseudo-clases** y **pseudo-elementos**. Debemos recordar que los pseudo-elementos van con doble dos puntos delante de la palabra y las pseudo-clases con dos puntos únicamente. *imagen 16*

Para comenzar con el ejemplo crearemos una lista.

```
<ul>
  <li></li>
  <li></li>
  <li></li>
  <li></li>
  <li></li>
  <li></li>
  </ul>
```

imagen 16



Ahora pasaremos a codificar los estilos para cambiar el aspecto de la lista. Primero aplicaremos algunos estilos al **body**

```
body{ background-image:url(bg.png);  
  background-color: hsla(0, 0%, 7%, 1);  
  font-family:Verdana, Geneva, sans-serif;  
  font-size:13px; color:#666;  
  counter-reset:circulo; }
```

Ahora voy a crear los círculos y el contador aplicando algunos estilos a la **etiqueta li** directamente y otros mediante el pseudo-elemento **:before**.

```
li{list-style-type: none; margin-top:25px; counter-increment:circulo; }  
li:before{ padding:5px; padding-left:9px; padding-right:9px;  
margin-right:20px; background-color:#CFF;  
border-radius: 20px 20px 20px 20px; content: counter(circulo);  
text-shadow:0 0 6px #000; box-shadow:2px 2px 5px #000; color:#000;}
```

De estos estilos destacamos **counter-increment: circulo**; que será nuestro contador incrementando en 1 cada uno de los **elementos li** de nuestra lista desordenada.

Después de esto nos queda únicamente añadir el texto para cada uno de los elementos de la lista y algunos estilos para que se muestren más elegantes en pantalla.

En este caso una sombra para el texto y un efecto **:hover** para que el texto cambie de color una vez

que pongamos nuestro ratón encima.

No debemos olvidar que el valor de esta lista se suele dar mediante código HTML.

```
li:after{content:"Ejemplo de lista de elementos";  
text-shadow:2px 2px 6px #000;}  
li:hover{color:#fff;}
```

Aplicando CSS a tablas

Ejemplo 1

Con el desarrollo de este ejemplo podremos profundizar el cambio de aspecto de una simple tabla para poderla convertir en un objeto más estético. A continuación se detalla el código html generando la estructura básica de la tabla. *imagen 15*

```
<table>  
<caption>Título de la tabla</caption>  
<tr>  
<th>...</th> <!-- esquina superior-izquierda -->  
<th>Columna 1</th> <!-- encabezado de columna 1 -->  
<th>Columna 2</th> <!-- encabezado de columna 2 -->  
<th>Columna 3</th> <!-- encabezado de columna 3 -->  
</tr>  
<tr>
```

```
<th>Fila 1</th>  <!-- encabezado de fila 1 -->
<td>celda 11</td>
<td>celda 12</td>
<td>celda 13</td>
</tr>
<tr>
<th>Fila 2</th>  <!-- encabezado de fila 2 -->
<td>celda 21</td>
<td>celda 22</td>
<td>celda 23</td>
</tr>
<tr>
<th>Fila 3</th>  <!-- encabezado de fila 3 -->
<td>celda 31</td>
<td>celda 32</td>
<td>celda 33</td>
</tr>
</table>
```

Veremos cómo aplicar los estilos a los distintos elementos de la tabla.

```
table { background: #f1f1f1; margin: 50px auto;
padding: 5px 10px; border: 1px solid #333333;
border-spacing: 8px; text-align: center;}

td { background: #f5f5f5; width: 100px;
border: 1px solid #999999; padding: 2px 4px;}
```

```
th { background: #D65600; padding: 4px;
  color: #ffffff; box-shadow: 1px 1px 2px #555555;}

caption { font: 1.2em/2 Verdana, sans-serif; color: #D65600;}

T2 { background: #f1f1f1; margin: 50px auto;
  padding: 0; border: 1px solid #c5c5c5;
  border-spacing: 1px; text-align: left; }

#T2 caption { font: 1.5em/2 Verdana, sans-serif; color: #999999; }

#T2 td { background: transparent; width: auto;
  border: none; padding: 0;}
#T2 th { background: #c5c5c5; padding: 0 8px;
  font: 0.8em Verdana, sans-serif; color: #ffffff;
  box-shadow: 1px 1px 1px #555555;}

#T2 td {text-align: right;}
#T2 tr:nth-child(2n+1) {background: #e1e1e1;}

tr:hover {color: #0000ff;}
#T2 td:hover {background: #666666; color: #ffffff;}
```

Ejemplo 2

Con este código repasaremos el uso de bordes redondeados y pseudo-elementos

```
table { border:1px black solid; border-radius:5px;
  min-width:400px; font-family: Helvetica,Arial;}
table td { padding:6px;}

/*Primera fila, primera celda*/
```

imagen 17

Título de la tabla

...	Columna 1	Columna 2	Columna 3
Fila 1	celda 11	celda 12	celda 13
Fila 2	celda 21	celda 22	celda 23
Fila 3	celda 31	celda 32	celda 33

Indice de Precios de Consumo

	2012M03	2012M02	2012M01	2012M03	2012M02	2012M01
General	101,055	100,38	100,275	0,7	0,1	-1,1
Alimentos y bebidas	101,574	101,472	101,357	0,1	0,1	0,3
Alcohólicas y tabaco	102,658	102,606	102,285	0,1	0,3	0,2
Vestido y calzado	95,131	91,604	93,232	3,9	-1,7	-14,4
Vivienda	102,154	101,908	101,729	0,2	0,2	0,5
Menaje	100,33	100,059	100,118	0,3	-0,1	-0,9
Medicina	97,704	97,709	97,648	0	0,1	-0,1

```

table tr:first-child td:first-child { border-radius:5px 0px 0px 0px;}

/*Primera fila, última celda*/
table tr:first-child td:last-child { border-radius:0px 5px 0px 0px;}

/*Última fila, primera celda*/
table tr:last-child td:first-child { border-radius:0px 0px 0px 5px;}

/*Última fila, última celda*/
table tr:last-child td:last-child { border-radius:0px 0px 5px 0px;}

```

Ejemplo 3

```

<!DOCTYPE HTML>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
  <title>Tabla de precios </title>
  <link rel="stylesheet" href="estilos.css">
</head>

<body>
  <div id="contenedor">
    <div id="tabla-precios">
      <div class="columnas">
        <ul>
          <li><strong>Selecciona tu plan</strong></li>

```

```
<li class="cabeza">    <h2>Plan 1</h2>
  <span>$10 / mes</span>
</li>
<li>100MB de Capacidad</li>
<li>1GB de Transferencia</li>
<li>Configuración gratuita</li>
<li>1 Cuenta FTP</li>
<li>1 Cuenta Email</li>
<li>1 Base de Datos</li>
<li>Dominio</li>
<li class="pie">
<a href="#" class="boton">Seleccionar</a>
</li>
</ul>
</div>
```

```
<div class="columnas">
<ul> <li><strong>Selecciona tu Plan</strong></li>
  <li class="cabeza"> <h2>Plan 2</h2>
    <span>$20 / mes</span>
  </li>
  <li>300MB de Capacidad</li>
  <li>3GB de Transferencia</li>
  <li>Configuración gratuita</li>
  <li>6 Cuenta FTP</li>
  <li>6 Cuenta Email</li>
```

```

    <li>6 Base de Datos</li>
    <li>Dominio</li>
    <li class="pie">
    <a href="#" class="boton">Seleccionar</a>
    </li>
  </ul>
</div>
<div class="columnas">
<ul>  <li><strong>Selecciona tu plan</strong></li>
    <li class="cabeza">  <h2>Plan 3</h2>
      <span>$40 / mes</span>      </li>
      <li>600MB de Capacidad</li>
      <li>6GB de Transferencia</li>
      <li>Configuración gratuita</li>
      <li>12 Cuenta FTP</li>
      <li>12 Cuenta Email</li>
      <li>12 Base de Datos</li>
      <li>Dominio</li>
      <li class="pie">
      <a href="#" class="boton">Seleccionar</a>
      </li>
    </ul>
</div>
<div class="columnas">
<ul>  <li><strong>Selecciona tu Plan</strong></li>
    <li class="cabeza">  <h2>Plan 4</h2>

```

```

    <span>$50 / mes</span> </li>
    <li>1GB de Capacidad</li>
    <li>12GB de Transferencia</li>
    <li>Configuración gratuita</li>
    <li>20 Cuenta FTP</li>
    <li>20 Cuenta Email</li>
    <li>20 Base de Datos</li>
    <li>Dominio</li>
    <li class="pie">
    <a href="#" class="boton">Seleccionar</a>
    </li>
  </ul>
</div>
</div>
</div>
</div>
</body>
</html>

```

Desarrollo de la hoja de Estilo.

```

* {margin: 0; padding: 0;}
body { background: #dfdede;}

#contenedor {width: 900px; margin: 80px auto; }
/*=====TABLA DE PRECIOS=====*/

```

```
#tabla-precios { font-family: Arial, Helvetica, sans-serif;
  height: 100%; font-size: 12px;  color: #787878;
  width: 100%; }
/*=====COLUMNAS=====*/
.columnas {float: left;  margin: 1px; width: 24%;
text-align: center;  position: relative; }

.columnas ul { border-bottom: 10px solid #222222;
  list-style-type: none; position: relative; }

.columnas ul li {
  background: #e6e6e6; border-bottom: 1px solid #b6b6b6;
  border-right: 1px solid #b6b6b6; padding: 1em .3em;
  color: #000000; position: relative; }

.columnas li:after {  color: #7a8e16;  content: '•';
  font-size: 25px;  left: 8px;  top: 2px;
  position: absolute; }

.columnas ul li:first-child {  background: #222222;
  color: #ffffff; padding: .5em .3em .8em;
  font-size: 13px; font-weight: bold;}

.columnas:first-child ul {border-left: 1px solid #b6b6b6;}
.columnas ul li:first-child:after {display: none;}
```

```
.columnas li:first-child {border-right: 1px solid transparent;}

.columnas .cabeza {
  background: #77948d; /* Old browsers */
  background: -moz-linear-gradient(top, #77948d 0%, #46635d 100%);
  background: -webkit-gradient(linear, left top, left bottom, color-stop(0%,#77948d), color-stop(100%,#46635d));
  background: -webkit-linear-gradient(top, #77948d 0%,#46635d 100%);
  background: -o-linear-gradient(top, #77948d 0%,#46635d 100%);
  background: -ms-linear-gradient(top, #77948d 0%,#46635d 100%);
  background: linear-gradient(to bottom, #77948d 0%,#46635d 100%);
  filter: progid:DXImageTransform.Microsoft.gradient(
startColorstr='#77948d', endColorstr='#46635d',GradientType=0 );
  display: block; border-right: 1px solid transparent;
  padding: 2em 0; }
.cabeza h2 {
  font-family: 'Allerta Stencil', sans-serif;
  text-transform: uppercase;
  font-size: 33px; line-height: 1.3em;
  color: #ffffff; }
.cabeza span {color: #ffffff; font-size: 12px;}
.cabeza:after {display: none;}

.columnas .pie {background: #fcfff4;
background: -moz-linear-gradient(top, #fcfff4 0%, #b3bead 100%);
background: -webkit-gradient(linear, left top, left bottom, color-
```

```
stop(0%,#fcfff4), color-stop(100%,#b3bead));
background: -webkit-linear-gradient(top,  #fcfff4 0%,#b3bead 100%);
background: -o-linear-gradient(top,  #fcfff4 0%,#b3bead 100%);
background: -ms-linear-gradient(top,  #fcfff4 0%,#b3bead 100%);
background: linear-gradient(to bottom,  #fcfff4 0%,#b3bead 100%);
  filter: progid:DXImageTransform.Microsoft.gradient(
startColorstr='#fcfff4', endColorstr='#b3bead',GradientType=0 );
padding: 2em 0; }
```

```
.columnas .pie:after {display: none; }
.pie .boton { background: #77948d;
  background: -moz-linear-gradient(top,  #77948d 0%, #46635d 100%);
background: -webkit-gradient(linear, left top, left bottom, color-
stop(0%,#77948d), color-stop(100%,#46635d));
background: -webkit-linear-gradient(top,  #77948d 0%,#46635d 100%);
background: -o-linear-gradient(top,  #77948d 0%,#46635d 100%);
background: -ms-linear-gradient(top,  #77948d 0%,#46635d 100%);
background: linear-gradient(to bottom,  #77948d 0%,#46635d 100%);
filter: progid:DXImageTransform.Microsoft.gradient( startColor-
str='#77948d', endColorstr='#46635d',GradientType=0 );
box-shadow: inset 0 -5px 0 #3a4845;
border-radius: 8px;  text-decoration: none;
padding: 1em 1.8em; color: #ffffff;  position: relative; }
.columnas:hover ul li.cabeza { background: #b8cf27;
  background: -moz-linear-gradient(top,  #b8cf27 0%, #7a8e16 100%);
```

```
background: -webkit-gradient(linear, left top, left bottom, color-
stop(0%,#b8cf27), color-stop(100%,#7a8e16));
background: -webkit-linear-gradient(top, #b8cf27 0%,#7a8e16 100%);
background: -o-linear-gradient(top, #b8cf27 0%,#7a8e16 100%);
background: -ms-linear-gradient(top, #b8cf27 0%,#7a8e16 100%);
background: linear-gradient(to bottom, #b8cf27 0%,#7a8e16 100%);
filter: progid:DXImageTransform.Microsoft.gradient(
startColorstr='#b8cf27', endColorstr='#7a8e16',GradientType=0 );
}
.columns: hover { -moz-transform: scale(1.15);
-webkit-transform: scale(1.15);
-o-transform: scale(1.15);
-moz-transition: all .2s linear 0s;
-webkit-transition: all .2s linear 0s;
-o-transition: all .2s linear 0s;
z-index: 5; }
.columns: hover ul li a.boton {
filter: progid:DXImageTransform.Microsoft.gradient(startColorstr='#b8
cf27', endColorstr='#7a8e16');
background: -ms-linear-gradient(top, #b8cf27, #7a8e16);
background: -moz-linear-gradient(top, #b8cf27, #7a8e16);
background: -o-linear-gradient(top, #b8cf27, #7a8e16);
background: -webkit-linear-gradient(top, #b8cf27, #7a8e16);
background: -webkit-gradient(linear, left top, left bottom, color-
```



```
stop(0, #b8cf27), color-stop(1, #7a8e16));
box-shadow:inset 0 -5px 0 #58600f; }

.columns:hover .boton:active {
  box-shadow:inset 0 -1px 0 #58600f; top: 3px;
  -moz-transition: all .2ms ease-in-out;
  -webkit-transition: all .2ms ease-in-out;
  -o-transition: all .2ms ease-in-out; }
```

imagen 18

Menú brillante

El cuerpo de este ejemplo es muy, codificaremos lo siguiente.

```
<body>
  <h1>Mi Sitio Web</h1>
  <a href="" class="opcion_iluminada">Blog</a>
  <a href="" class="opcion_iluminada">Quiénes somos</a>
  <a href="" class="opcion_iluminada">Contacto</a>
</body>
```

Los estilos creados son los siguientes:

```
body { background: #444; }
```

imagen 18

Selecciona tu Plan	Selecciona tu Plan	Selecciona tu Plan	Selecciona tu Plan
PLAN 1 \$10 / mes	PLAN 2 \$20 / mes	PLAN 3 \$40 / mes	PLAN 4 \$50 / mes
• 100MB de Capacidad	• 300MB de Capacidad	• 600MB de Capacidad	• 1GB de Capacidad
• 1GB de Transferencia	• 3GB de Transferencia	• 6GB de Transferencia	• 12GB de Transferencia
• Configuración gratuita	• Configuración gratuita	• Configuración gratuita	• Configuración gratuita
• 1 Cuenta FTP	• 6 Cuenta FTP	• 12 Cuenta FTP	• 20 Cuenta FTP
• 1 Cuenta Email	• 6 Cuenta Email	• 12 Cuenta Email	• 20 Cuenta Email
• 1 Base de Datos	• 6 Base de Datos	• 12 Base de Datos	• 20 Base de Datos
• Dominio	• Dominio	• Dominio	• Dominio
Seleccionar	Seleccionar	Seleccionar	Seleccionar

```
h1 {    font-size: 38px;    font-style: italic;
    color: #333;    padding: 10px;
    text-shadow: -1px -1px 0px #101010, 1px 1px 0px #505050;
}

.opcion_iluminada {    font-size: 20px;    color:#999;
    text-decoration:none;    padding: 10px;  }

.opcion_iluminada:hover {    color:#fff;
    text-shadow: 0px 0px 15px #fff;  }
</style>
```

Un dato importante, para que las sombras puedan verse, el color de fondo, en este caso de la página (elemento body, propiedad background), no puede ser negro porque entonces no se vería la sombra. Nosotros lo hemos establecido a **#444**, un gris muy oscuro, pero sin llegar a ser negro.

El efecto de bajo relieve del título (**h1**) lo conseguimos dándole un color más oscuro que el fondo (**#333**) y aplicando dos sombras, separadas por una coma, en la definición de la propiedad **text-shadow**:

- | Sombra más oscura en la parte superior e izquierda: -1px -1px 0px #101010
- | Sombra más clara en la parte inferior y derecha: 1px 1px 0px #505050

Respecto al menú, cada opción tiene asignada el estilo **opcion_iluminada** y cuando el puntero del ratón se sitúa encima se aplica el estilo **opcion_iluminada:hover**. Este es un ejemplo muy sencillo, el menú se puede construir de otras formas.

En el estilo de **opcion_iluminada** (cuando el puntero del mouse no está sobre la opción), simplemente hemos establecido el color del texto a **#999**, un gris.

De esta forma, cuando el puntero del mouse se sitúe encima de la opción, se aplicará el estilo de **opcion_iluminada: hover** y el color del texto será más intenso, concretamente blanco (#fff). Esto ya dará la sensación de que la opción se ilumina, pero el resultado no estará muy logrado. Lo vamos a mejorar añadiendo una sombra blanca para crear el efecto de resplandor mediante la propiedad **text-shadow** y el valor:

| 0px 0px 15px #fff

Los valores deberán ajustarse según el tamaño del texto y los enlaces del menú. *imagen 19*

Como alternativa de este ejercicio y de este menú pasaremos a desarrollar otro ejemplo a continuación. Comenzaremos a codificar el menú con una lista adentro de un contenedor, puedes desarrollar en mismo en la zona de <nav>

```
<div id="menu">
  <ul>
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Conecta</a></li>
    <li><a href="#">Descubre</a></li>
    <li><a href="#">Cuenta</a></li>
```

imagen 19





```
</ul>
</div>
```

Los estilos están formados de la siguiente manera:

```
#menu{ background-color: #fff;
height: 41px; overflow: hidden;
padding-left: 100px;}

#menu ul{ padding: 0; margin: 0; }

#menu ul li{ border-bottom: solid 10px orange;
display: inline-block; list-style: none;
padding: 10px 0 20px 0; -webkit-transition: all 0.3s;
-moz-transition: all 0.3s; transition: all 0.3s;}

#menu ul li:hover{ padding: 10px 0 10px 0; }

#menu ul li a{ color: black; padding: 10px 25px;
text-decoration: none; }
```

Menú Multinivel

Trabajaremos con una lista con clase de “**nav**”, que almacenara los ítems del menú:

```

<!DOCTYPE HTML>
<head>
  <title>Menu desplegable</title>
  <link rel="stylesheet" href="estilos.css" />
</head>
<body>
<ul class="nav">
  <li><a href="">Home</a></li>
  <li><a href="">Servicios</a></li>
  <li><a href="">Acerca</a></li>
  <li><a href="">Contacto</a></li>
</ul>
</body>
</html>

```

Los submenús se desplegaran cuando el usuario pase el cursor, estos irán como listas dentro de listas(**ul > li > ul**). Este ejemplo funcionara con cualquier cantidad de submenús, eso quiere decir que puedes incluir los niveles de menús que quieras, en este caso tendremos 3 niveles:

```

<ul class="nav">
  <li><a href="">Home</a></li>
  <li><a href="">Servicios</a>
    <ul>
      <li><a href="">Diseno grafico</a></li>
      <li><a href="">Diseno web</a>
        <ul>

```

```
<li><a href="">Submenu 1</a></li>
<li><a href="">Submenu 2</a></li>
<li><a href="">Submenu 3</a></li>
<li><a href="">Submenu 4</a></li>
<li><a href="">Submenu 5</a></li>
</ul>
</li>
<li><a href="">Marketing</a>
<ul>
<li><a href="">Submenu 1</a></li>
<li><a href="">Submenu 2</a></li>
<li><a href="">Submenu 3</a>
<ul>
<li><a href="">Submenu 1</a></li>
<li><a href="">Submenu 2</a></li>
<li><a href="">Submenu 3</a></li>
<li><a href="">Submenu 4</a></li>
</ul>
</li>
</ul>
</li>
<li><a href="">SEO</a></li>
</ul>
</li>
```

```
<li><a href="">Acerca</a>
  <ul>
    <li><a href="">Historia</a></li>
    <li><a href="">Mision</a></li>
    <li><a href="">Vision</a></li>
  </ul>
</li>
<li><a href="">Contacto</a></li>
</ul>
```

Comenzaremos a definir los estilos para este menú, recuerda de enlazarlo en forma externa como se ve al comienzo de esta clase. Empezaremos con un reset básico y algo de decoración para el menú:

```
* { font-family:sans-serif; list-style:none;
text-decoration:none; margin:0; padding:0;}

.nav > li {float:left;}

.nav li a { background:#0c9ba0; color:#FFF;
display:block; border:1px solid; padding:10px 12px; }

.nav li a:hover { background:#0fbfc6; }
```

Se especifica que solo los **li** que sean descendientes directos del primer **ul** tengan **float: left**, esto es para que solo el menú principal sea horizontal y los submenús se mantengan en vertical: *imagen 20*

Por defecto todos los submenús no serán visibles, los ocultaremos usando **display: none**.

```
.nav li ul { display:none; position:absolute; min-width:140px; }
```

Todos los submenús tendrán un ancho mínimo de 140px para que no se vean desiguales y llevaran **position: absolute** para que no afecten el ancho del menú principal. Proseguimos con la parte que hará que se muestre el submenú oculto:

```
.nav li:hover > ul{display: block; }
```

En este código le estamos indicando que cuando el cursor pase sobre cualquier **li** su descendiente **ul** se muestre (display: block). *imagen 21*

El problema ahora es que los submenús de segundo nivel en adelante se están mostrando pero no como deberían: *imagen 22*

Lo que tenemos que hacer es que estos se muestren a la derecha de su respectivo ancestro **li**:

```
.nav li ul li {position:relative; }  
.nav li ul li ul { right:-140px; top:0; }
```

imagen 20

Home	Servicios	Acerca	Contacto
	Diseno grafico	Historia	
	Diseno web	Mision	
	Submenu 1	Vision	
	Submenu 2		
	Submenu 3		
	Submenu 4		
	Submenu 5		
	Marketing		

imagen 21

Home	Servicios	Acerca	Contacto
	Diseno grafico		
	Diseno web		
	Marketing		
	SEO		

Los submenús de segundo nivel tendran **right: -140px**, para empujarlos hacia la derecha, es importante notar que este valor es el mismo que el que definimos anteriormente como ancho mínimo, y además tendrá **top: 0** esto es para que se posicione al mismo nivel que su ancestro **li** que tiene **position: relative**.

Esto afectara todos los submenús de segundo nivel en adelante: *imagen 23*

Añadir un indicador a los elementos con submenú

Lo primero es añadir el elemento que hará de indicador al lado de cada enlace del menú, en este caso estamos usando una flechita (si prefieres usar una imagen también funcionara) su código HTML es **▼**:

```
<ul class="nav">
  <li>
    <a href="#">Home<span class="flecha">&#9660;</span></a>
  </li>
  <li>
    <a href="#">Servicios<span class="flecha">&#9660;</span></a>
  </li>
  <li>
    <a href="#">Diseno grafico<span class="flecha">&#9660;</span></a>
  </li>
  <li>
    <a href="#">Diseno web<span class="flecha">&#9660;</span></a>
  </li>
</ul>
```

imagen 22

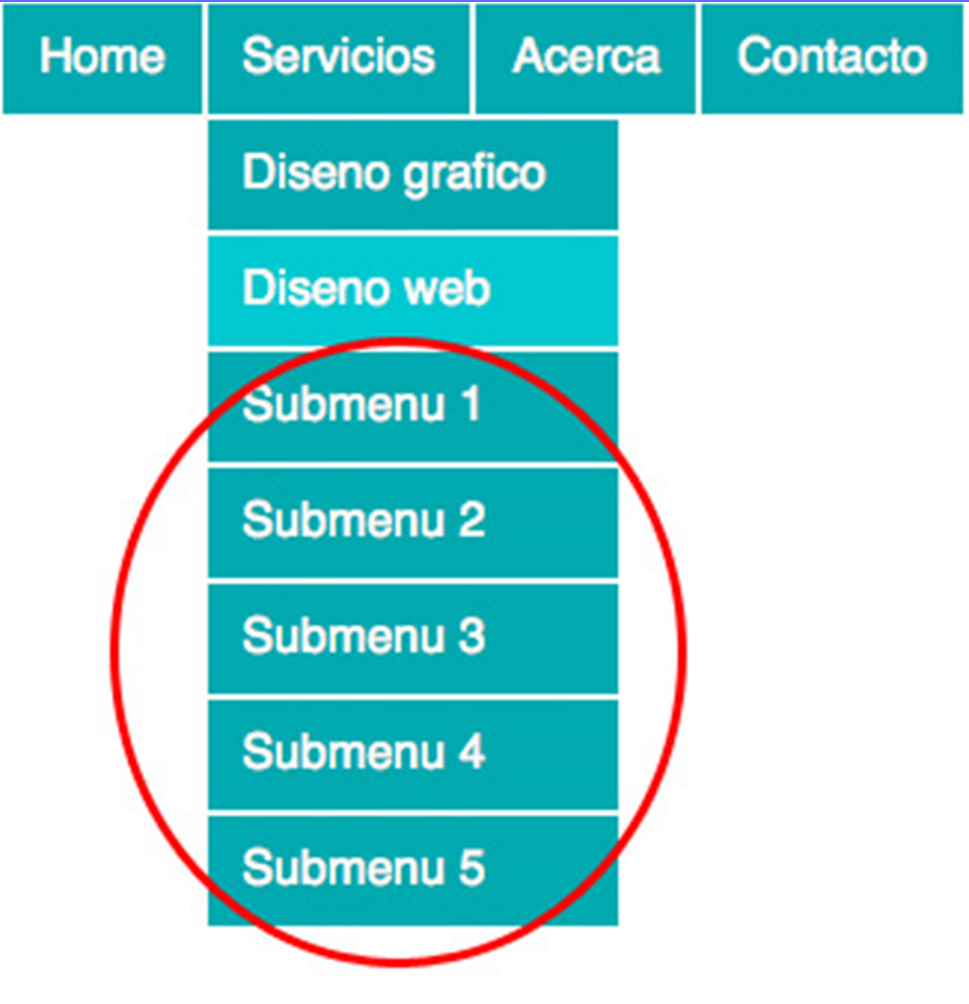


imagen 23





```
<li><a href="#">Submenu 1<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 2<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 3<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 4<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 5<span class="flecha">&#9660</span></a></li>
</ul>
</li>
<li>
<a href="#">Marketing<span class="flecha">&#9660</span></a>
<ul>
<li><a href="#">Submenu 1<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 2<span class="flecha">&#9660</span></a></li>
<li>
<a href="#">Submenu 3<span class="flecha">&#9660</span></a>
<ul>
<li><a href="#">Submenu 1<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 2<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 3<span class="flecha">&#9660</span></a></li>
<li><a href="#">Submenu 4<span class="flecha">&#9660</span></a></li>
</ul>
</li>
</ul>
</li>
<li><a href="#">SEO<span class="flecha">&#9660</span></a></li>
</ul>
</li>
```





```
<li>
<a href="#">Acerca<span class="flecha">&#9660</span></a>
<ul>
<li><a href="#">Historia<span class="flecha">&#9660</span></a></li>
<li><a href="#">Mision<span class="flecha">&#9660</span></a></li>
<li><a href="#">Vision<span class="flecha">&#9660</span></a></li></ul>
</li>
<li>
<a href="#">Contacto<span class="flecha">&#9660</span></a>
</li>
</ul>
```

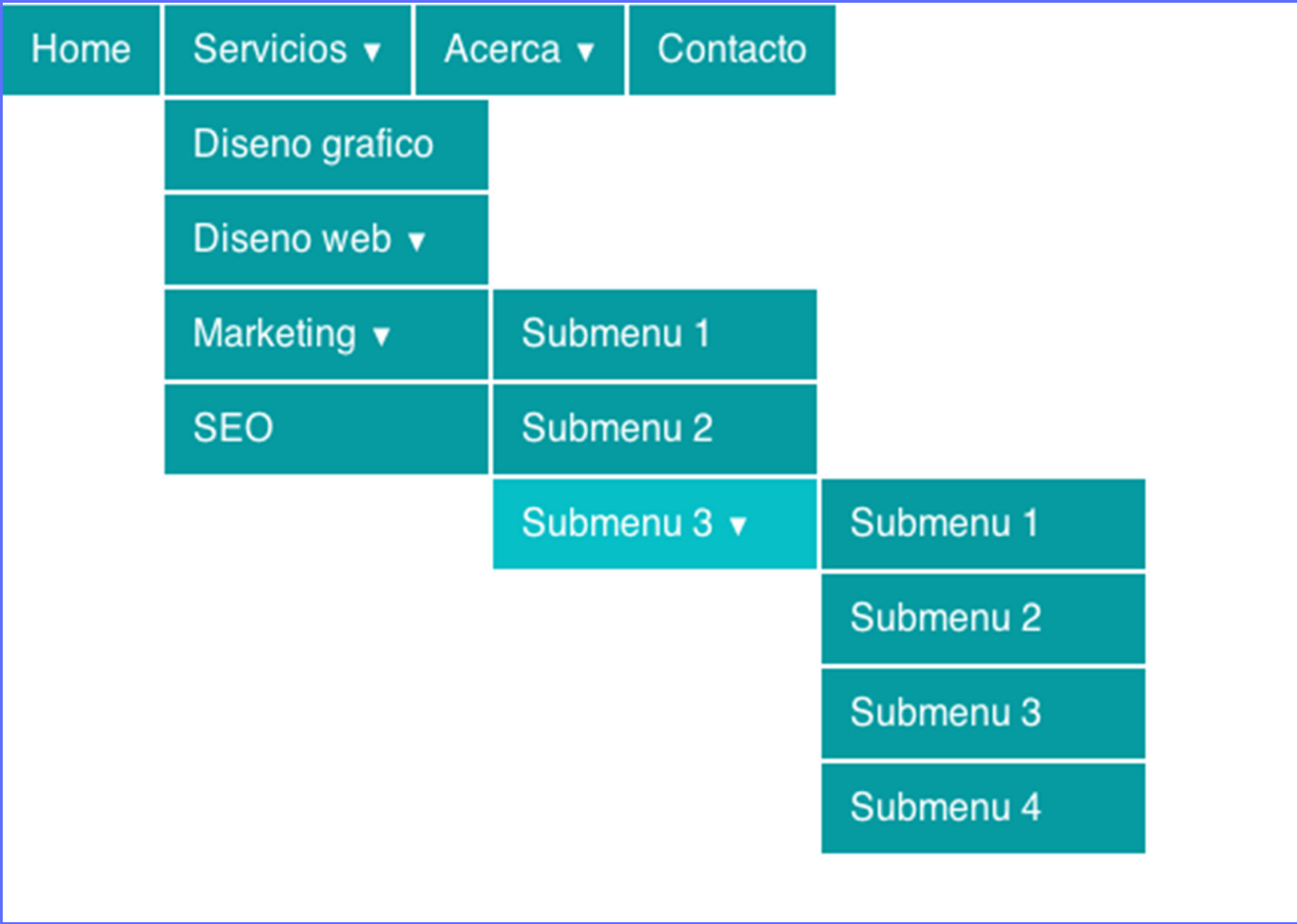
Ahora lo que haremos será definir que por defecto esta flecha este oculta en todos los elementos del menú:

```
.nav li .flecha{font-size: 9px;
padding-left: 6px; display: none; }
```

También le reducir el tamaño de la fuente y lo aleje un poco del texto para que se vea bien. La parte esencial de esto es la siguiente: vamos a usar la psuedo-clase CSS3 **:not** en combinación con **:last-child** para excluir a todos los elementos del menú que no contengan submenús (todos los **<a>** que no sean **:last-child**) y solo afectar a los **span.flecha** de los que sí: *imagen 24*

```
.nav li a:not(:last-child) .flecha {
display: inline;}
```

imagen 24



Menú vertical

Siguiendo las mismas técnicas para la construcción de un menú, te presentamos en este ejercicio, donde se muestra un menú vertical con efecto en cada una de sus opciones.

```
<html>
<style>

.shadowblockmenu-v{font: bold 14px Germand;
width: 190px; }

.shadowblockmenu-v ul{ border: 1px solid #eee;
padding: 0; margin: 0; list-style: none;}

.shadowblockmenu-v ul li{ margin:0; padding:0;}

.shadowblockmenu-v ul li a{
display:block; text-transform: uppercase;
color: #494949; padding: 10px 15px;
text-decoration: none; border-bottom: 1px solid #cacaca;
border-right: 1px solid #cacaca;
-moz-box-shadow: inset 7px 0 10px rgba(114,114,114, 0.6);
-webkit-box-shadow: inset 7px 0 10px rgba(114,114,114, 0.6);
box-shadow: inset 7px 0 10px rgba(114,114,114, 0.6);
text-shadow: 0 -1px 1px #cfcfcf;
-moz-transition: all 0.2s ease-in-out;
```

```
-webkit-transition: all 0.2s ease-in-out;
-o-transition: all 0.2s ease-in-out;
-ms-transition: all 0.2s ease-in-out;
transition: all 0.2s ease-in-out;
}

.shadowblockmenu-v ul li a:hover, .shadowblockmenu-v ul li a.selected{
color: black;
-moz-box-shadow: inset 7px 0 10px rgba(216,89,39, 0.5), inset 0 0 15px
rgba(216,89,39, 0.6), inset 0 0 20px rgba(216,89,39, 0.8);
-webkit-box-shadow: inset 7px 0 10px rgba(216,89,39, 0.5), inset 0 0
15px rgba(216,89,39, 0.6), inset 0 0 20px rgba(216,89,39, 0.8);
box-shadow: inset 7px 0 10px rgba(216,89,39, 0.5), inset 0 0 15px
rgba(216,89,39, 0.6), inset 0 0 20px rgba(216,89,39, 0.8);
}

</style>

</style>
<body>
<div class="shadowblockmenu-v">
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">Nosotros</a></li>
<li><a href="#">Servicios</a></li>
<li><a href="#">Galeria</a></li>
```

```
<li><a href="#">Chat</a></li>
<li><a href="#">Contacto</a></li>
</ul>
</div>
</body>
</html>
```

Efecto en imágenes

Ejemplo 1

Comenzaremos con el código html

```
<ul class="contenedor caption">
  <li>
    <figure>
      <div></div>
      <figcaption>
        <h3>Poyecto 1</h3>
        <span>
          Lorem Ipsum is simply dummy text of the printing and
typesetting industry.
        </span>
        <a href="" class="btn_hover">Visitar</a>
      </figcaption>
    </figure>
```



```
</li>
```

```
</ul>
```

El código HTML es básico y fácil de entender, tenemos una lista `<ul>` asociada a dos clases dentro tenemos los respectivos `<li>` dentro tenemos la etiqueta `<figure>` de html5 que es la encargada de contener todo el contenido a mostrar, después tenemos un `<div>` y adentro mostramos la imagen `<img>`, aparte tenemos la etiqueta `<figcaption>` donde mostraremos la información adicional es decir el caption, adentro tenemos un título en `<h3>` un `<span>` para mostrar el texto y un enlace `<a>`. Definimos los estilos utilizados:

```
.contenedor {    padding: 20px 20px 100px 20px;
    max-width: 1200px;    margin: 0 auto;
    list-style: none;    text-align: center; }
```

```
.contenedor li {
    display: inline-block;    width: 363px;
    margin: 0;    padding: 20px;    text-align: left;
    position: relative; }
```

A los `<ul>` y el/los `<li>`, le damos las proporciones y dimensiones en las que se van mostrar, los `<li>` que al final será cada imagen la mostramos con **display: inline-block** por si tenemos muchas se ubicaran una al lado de la otra, con un ancho de 363px para que encajen de a tres por líneas en caso de tener varias.

```
.contenedor figure {    margin: 0;    position: relative; }

.contenedor figure img {    max-width: 100%;
    display: block;    position: relative;}

.contenedor figcaption {    position: absolute;
    top: 0;    left: 0;    padding: 20px;
    background: #666666;    color: #ed4e6e;}

figcaption span {
    color: #DDDDDD;    display: block;
    font-size: 13px;    margin: 12px 0;}

.contenedor figcaption h3 {    border-bottom: 1px solid #777777;
    color: #FFFFFF;    margin: 0;    padding: 0 0 5px;}

.contenedor figcaption a {    text-align: center;
    padding: 5px 10px;    border-radius: 2px;
    display: inline-block;    background: #31a7c4;
    color: #fff; }
```

Aplicamos los estilos de cada elemento que compone el ítem, es decir, la etiqueta figure, la imagen, la etiqueta **figcaption** el texto que está dentro de un **span**, el **h3** y el enlace, estos son estilos para que se vean bien y para posicionarlos, hasta ahora esto es lo que vemos.

Aplicamos un poco de perspectiva con la propiedad **perspective** y **perspective-origin**, con esto determinamos la intensidad del efecto 3d y cambiar la posición de origen interior del elemento que tendrá efecto 3d.

```
.caption li {  -webkit-perspective: 1800px;
               -moz-perspective: 1800px;    perspective: 1800px;
               -webkit-perspective-origin: 0 50%;
               -moz-perspective-origin: 0 0%;    perspective-origin: 0 0%;}
```

Básicamente le decimos al navegador que estamos trabajando con características 3d y por ahora ocultamos el div que tenemos dentro de figure

```
.caption figure {
  -webkit-transform-style: preserve-3d;
  -moz-transform-style: preserve-3d;
  transform-style: preserve-3d;
}
.caption figure > div {
  overflow: hidden;
}
```

Aplicamos animación a la imagen dentro del ítem, le decimos que se mueva 23% de su ubicación en el eje x con la propiedad **transla-teX**, esta se moverá de izquierda a derecha y tendrá una transición que aplicamos con la propiedad **transition**.

```
.caption figure img {
  -webkit-transition: -webkit-transform 0.3s ease-out;
  -moz-transition: -moz-transform 0.3s ease-out;
  transition: transform 0.3s ease-out;
}
.caption figure:hover img
{
  -webkit-transform: translateX(23%);
  -moz-transform: translateX(23%);
  -ms-transform: translateX(23%);
  transform: translateX(23%);
  cursor: pointer;
}
```

Ajustamos la altura al 100% de nuestro caption y lo dejamos invisible con la propiedad **opacity** en 0, con **backface-visibility: hidden** nos aseguramos que al momento de girar el elemento en efecto 3d la otra cara del elemento quede oculta ya que lo estamos rotando -90 grados y en su punto de origen lo dejamos en 0 0. Luego en el estado **hover**, es decir cuando situé el mouse sobre él lo hacemos totalmente visible cambiando su opacidad, cambiamos la rotación y aplicamos la transición a todos los cambios para dar la sensación de movimiento.

```
.caption figcaption {
  height: 100%;    width: 50%;
  opacity: 0;      -webkit-backface-visibility: hidden;
  -moz-backface-visibility: hidden;
  backface-visibility: hidden;
  -webkit-transform-origin: 0 0;
  -moz-transform-origin: 0 0;
  transform-origin: 0 0;
  -webkit-transform: rotateY(-90deg);
  -moz-transform: rotateY(-90deg);
  transform: rotateY(-90deg);
  -webkit-transition: -webkit-transform 0.4s, opacity 0.1s 0.3s;
  -moz-transition: -moz-transform 0.4s, opacity 0.1s 0.3s;
  transition: transform 0.4s, opacity 0.1s 0.3s;
}

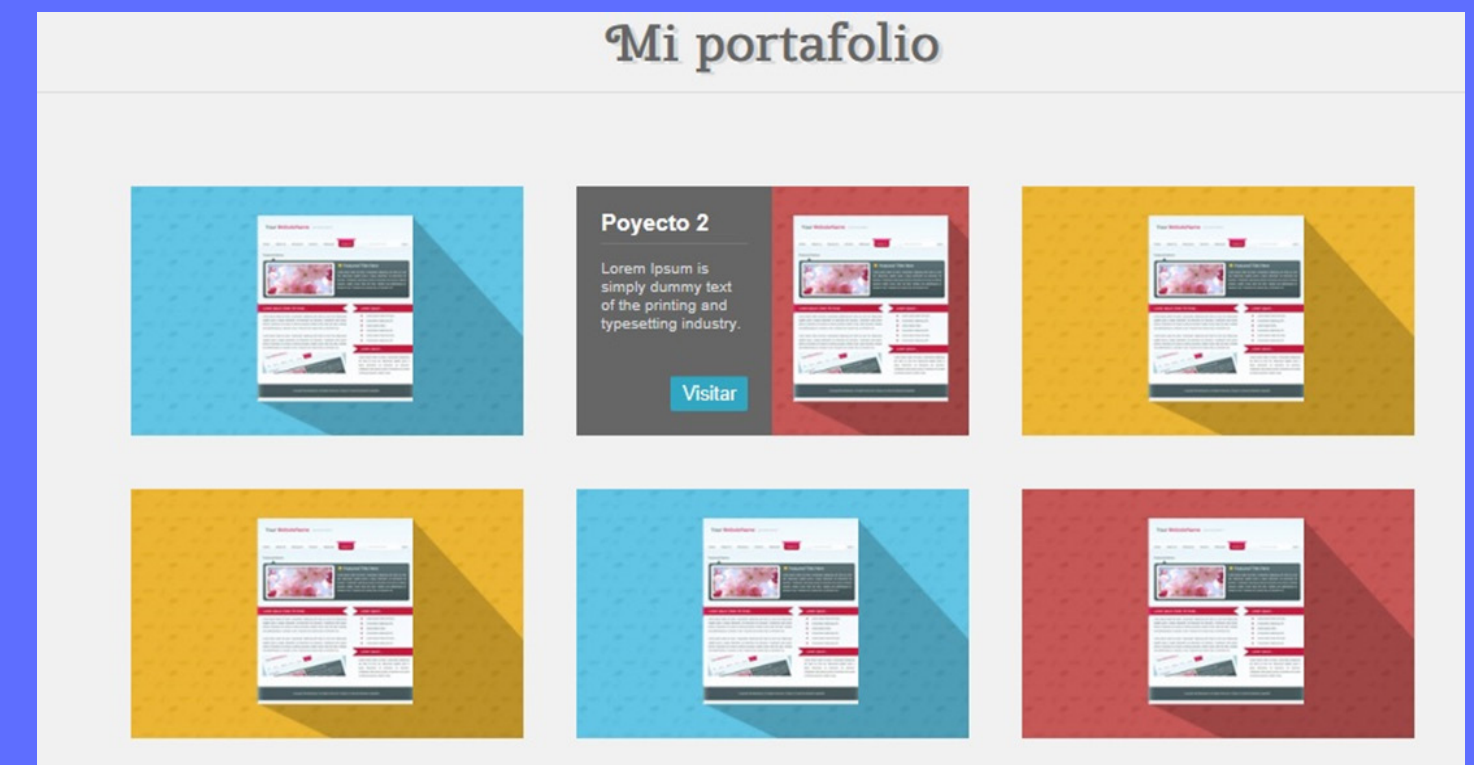
.caption figure:hover figcaption{
  opacity: 1;      -webkit-transform: rotateY(0deg);
  -moz-transform: rotateY(0deg);
  transform: rotateY(0deg);
  -webkit-transition: -webkit-transform 0.4s, opacity 0.1s;
  -moz-transition: -moz-transform 0.4s, opacity 0.1s;
  transition: transform 0.4s, opacity 0.1s;
}
```

Y finalmente aplicamos los estilos al enlace que está dentro del **figcaption** que al final será un botón que tendrá un enlace, lo posicionamos absolutamente y aplicamos estilos de **background** y transición en el estado **hover**.

```
.caption figcaption a {  
    position: absolute;  
    bottom: 20px;    right: 20px; }  
  
.btn_hover:hover {  
    background: #3fc7e8;  
    -webkit-transition: all 0.2s ease-in;  
    -moz-transition: all 0.2s ease-in;  
    -o-transition: all 0.2s ease-in;  
}
```

Y eso es todo, tenemos listo nuestro efecto caption con css3 para utilizar en proyectos, para mostrar nuestro portafolio personal, mostrar información de productos etc. *imagen 25*

imagen 25



Transiciones CSS3

Las transiciones con CSS, por si no lo sabemos, son efectos o animaciones que se aplican a los elementos al pasar de un estilo a otro. Con estas transiciones podemos suavizar el cambio de estilos sin la necesidad de utilizar javascript o flash.

Antes de nada, indicar que para aplicar transiciones debemos tener en cuenta varias cosas: **transition-property**.

Comenzaremos a construir los estilos, para este ejemplo:

```
body { background-color:#333;
background-color:hsla(0, 0%, 20%, 1);
background-image:
  url(bg3.png),
  url(bg2.png),
  url(bg.png);

background-size:auto;
margin-top:50px;
margin-left:300px; }

.texto{
  font-family:Tahoma, Geneva, sans-serif;
```

```
font-size:15px; font-weight:bold;
text-shadow: -1px 1px 1px hsla(180, 1%, 30%, 1);
letter-spacing:5px; color:#222; }
```

Ahora crearemos un marco para introducir unos botones:

```
.marco{
  background-color:hsla(250, 1%, 22%, 1);
  background-image:url(bg-marco.png);
  box-shadow: -1px 1px 1px hsla(180, 1%, 30%, 1);
  height:25px; width:400px; padding:20px;
  border: hsla(0, 0%, 0%, 1) 2px solid;

  border-radius:5px 5px 5px 5px;
  margin-top:20px; margin-bottom:20px; }
```

Ahora crearemos en botón al que le aplicaremos las transiciones:

```
.boton{
  background-color:hsla(180, 1%, 18%, 1);
  width:15px; height:17px; float:left;
  border-radius:5px 5px 5px 5px;
  padding-left:5px;padding-top:3px;
  box-shadow: -1px 5px 5px hsla(180, 1%, 10%, 1);
  border-top: hsla(180, 1%, 15%, 1) thin solid;
  border-left: hsla(180, 1%, 12%, 1) thin solid;
```



```
border-right: hsla(180, 1%, 9%, 1) thin solid;
border-bottom: hsla(180, 1%, 6%, 1) thin solid;

-webkit-transition-property:background-color, box-shadow;
-webkit-transition-duration:0.4s;
-webkit-transition-timing-function:ease-in;
-webkit-transition-delay:0s;
}
```

Tenemos que tener en cuenta que muchas de estas propiedades todavía no funcionan de una forma correcta en todos los navegadores, por eso debemos emplear los prefijos para los navegadores CSS3. Definiremos el estado hover del botón:

```
.boton:hover{
background-color:hsla(180, 1%, 15%, 1);
box-shadow: 1px 1px 15px hsla(51, 23%, 95%, 0.1);
float:left;
border-top: hsla(180, 1%, 2%, 1) thin solid;
border-left: hsla(180, 1%, 2%, 1) thin solid;
border-right: hsla(180, 1%, 2%, 1) thin solid;
border-bottom: hsla(180, 1%, 5%, 1) thin solid;
-webkit-transition-property:background-color, box-shadow;
-webkit-transition-duration:0.4s;
-webkit-transition-timing-function:ease-out;
-webkit-transition-delay:0s;
}
```

Después de tener ya todo creado, puedes incrementar el número de botones en el marco para generar un lindo menú.

Efecto Luz de Neón

Con el siguiente ejemplo trabajaremos las sombras, transiciones y fondo de manera tal que genere el famoso efecto luz de neón, el código se muestra a continuación:

```
<html>
<style>

*{ padding:0; margin:0; }
body{ background:#222 url('tronfondo.jpg') repeat; }
.luz-superior{ background-color:#fff;
  -webkit-box-shadow: 0px 0px 40px rgb(8, 213, 248),
    inset 0px 0px 10px rgb(8, 213, 248),
    inset 0px 0px 20px rgb(8, 213, 248),
    inset 0px 0px 30px rgb(8, 213, 248);
}
.luz-interior{
  background:#222 url('tronfondo.jpg') repeat;
  -webkit-box-shadow:
    0px 0px 10px rgb(8, 213, 248),
    0px 0px 20px rgb(8, 213, 248),
    0px 0px 30px rgb(8, 213, 248),
    0px 0px 40px rgb(8, 213, 248),
```

```
    inset 0px 0px 40px rgb(12, 195, 200);
}
.capa1{ width:200px; height:200px;
-webkit-border-radius:50%; position:relative; }

.capa2{ width:175px; height:175px;
-webkit-border-radius:50%; position:absolute;
top:12px; left:12px; }

.capa3{ width:120px; height:120px;
-webkit-border-radius:50%; position:absolute;
top:27px; left:27px; }

.capa4{ width:90px; height:90px;
-webkit-border-radius:50%; position:absolute;
top:15px; left:15px; }

.uno{ position:absolute; top:160px;
left:180px; }

.cola{ width: 210px; height:200px;
background-color:rgba(41, 209, 238, 0.28);
position:absolute; top:160px; left: 160px;
-webkit-border-radius:50%; }
```



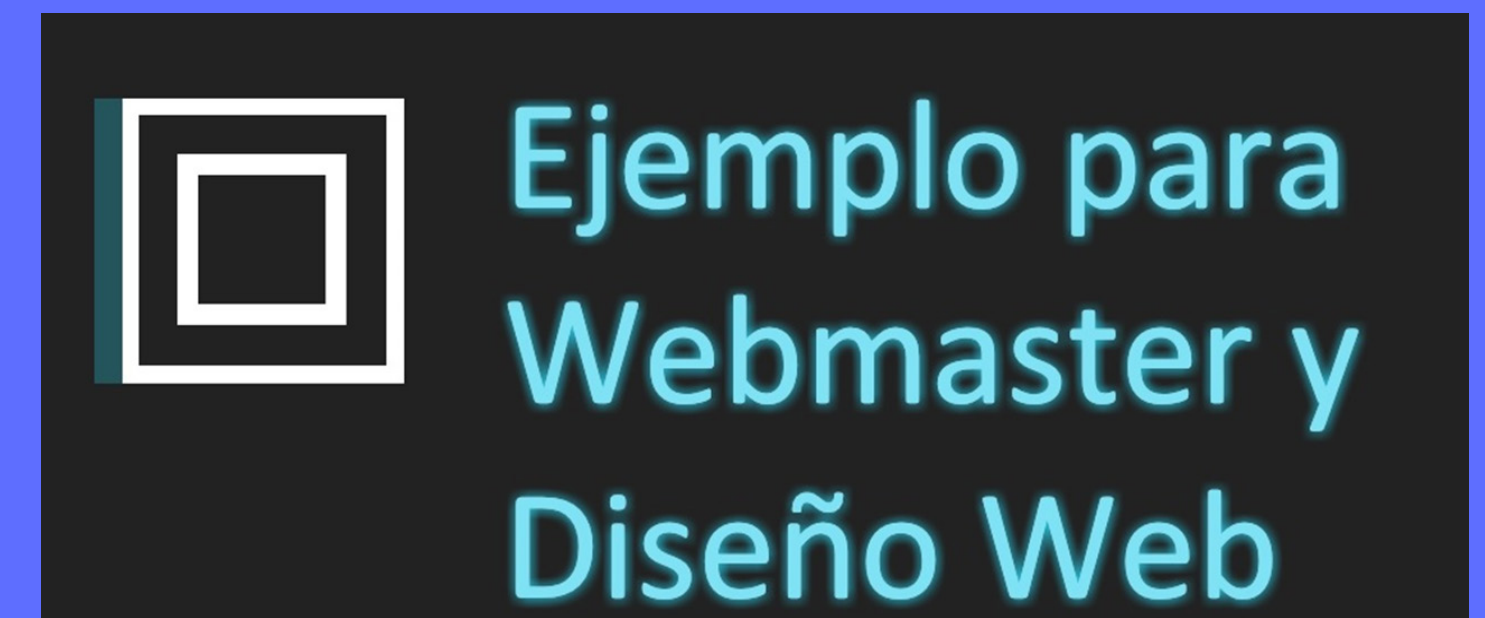
```
.nombre{  font-family: 'calibri';
  color:rgba(41, 209, 238, 0.6);    width:800px;
height: 200px;  font-size:7em;
text-shadow:
  0 0 1px #fff,
  0 0 10px rgb(41, 209, 238);
position:absolute;  top:130px;  left:450px;  }

</style>
<body>
<div class="cola luz-superior">  </div>
  <div class="uno capa1 luz-superior">
    <div class="capa2 luz-interior">
      <div class="capa3 luz-superior">
        <div class="capa4 luz-interior"></div>
      </div>
    </div>
  </div>

  <div class="nombre">Ejemplo para Webmaster y Diseño Web</div>
</body>
</html>
```

Imagen 26

imagen 26



Formatear zonas de Html5

Comenzaremos a desarrollar el siguiente código en HTML5.

```
<!DOCTYPE html/>
<html lang="es">
<head>
  <meta charset="utf-8" />    <title> HTML5 y CSS3    </title>
  <link rel="stylesheet" href="stylo.css" />
</head>
<body>
  <header>
    <h1>Esta es la cabecera de nuestro sitio web</h1>
  </header>
  <nav>    Esta es nuestra botonera    </nav>
  <section id="contenedor">
    <section id="principal">
      Esto es una seccion
      <article>    Esto es un article    </article>
    </section>
    <aside>  Esto es un aside    </aside>
  </section>
  <footer>
    Este es el pie de nuestra p&aacute;gina
  </footer>
</body>
</html>
```

A continuación comenzaremos a definir los estilos seteando la zona de trabajo con márgenes:

```
*{ margin: 0; padding: 0; }
```

Ahora establecemos la tipografía de nuestro sitio web para eso tenemos que utilizar la el atributo de CSS3 que es **@font-face** y lo que les vamos a establecer a este atributo son 2 mandatos que el tipo de letras y el estilo sea normal, esta es otra alternativa para trabajar con fuentes.

```
@font-face {font-weight:normal; font-style:normal;}

body{ background: #FFF; color:#F60;
font-family:Helvetica, Verdana;
font-size: 2em; }

h1{ color:#F60; margin:0.25em auto;
text-align:center;
text-shadow: 5px 5px 10px rgba(255,255,255,0.5); }
```

imagen 27

imagen 27



En este código lo primero que establecimos fue el color de las letras del <h1> en este caso (color:#F60;) que corresponde al naranja, luego el margin es espacio que va a tener de los demás elementos. También establecimos el text-align:center; esto es para alinear el texto al centro y algo muy importante que no se puede quedar es el atributo (text-shadow:) *Imagen 28*

Nuestro código para el <header> seria el siguiente.

```
header { background:#DEDEDE;  
border-radius:0.5em; margin: 0.5em auto;  
max-width:960px; padding:0.25em; text-align:center; }
```

imagen 29

```
header ,nav,footer{ background:#DEDEDE;  
border-radius:0.5em; margin: 0.5em auto;  
max-width:960px; padding:0.25em; text-align:center; }
```

imagen 30

El código CSS3 para el section contenedor sería el siguiente:

```
section#contenedor{ margin:0.2em auto; max-width:960px;  
padding:0; text-align:center; }
```

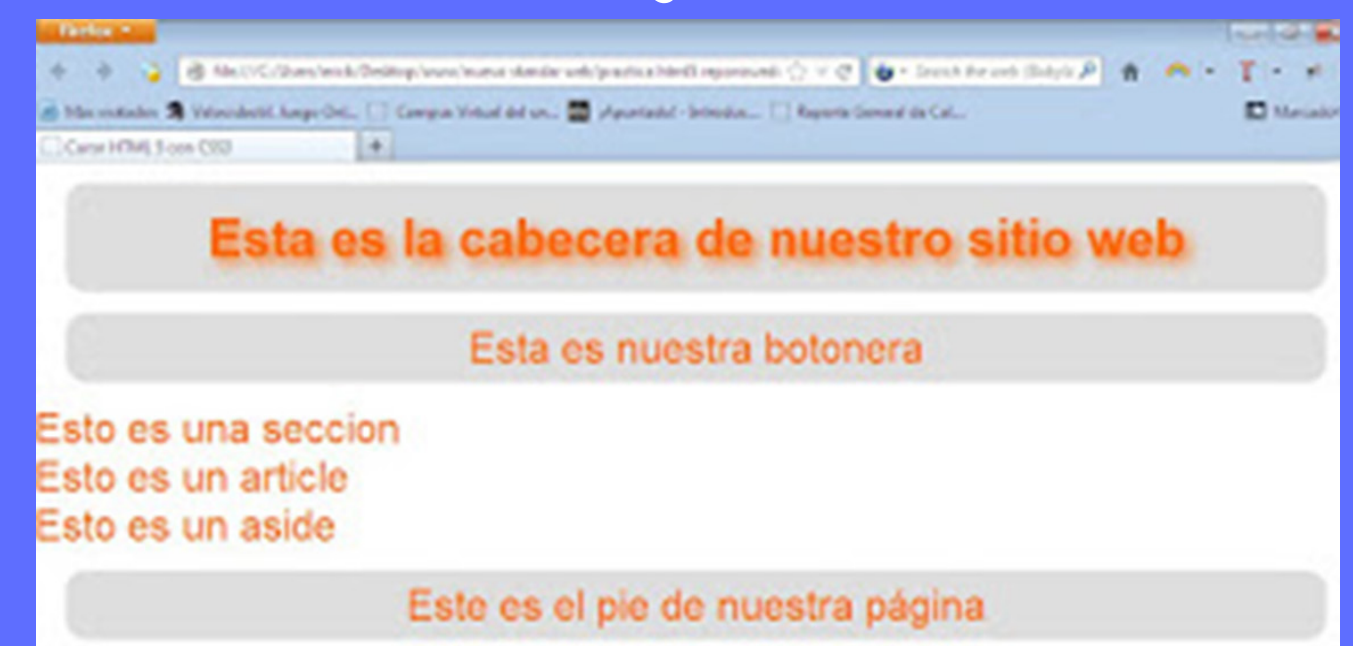
imagen 28



imagen 29



imagen 30



Ahora seguimos con el **section#principal**. A este le decimos que tenga un **background** o color de fondo un **border-radius** para que tenga border curvos, y otro atributo que añada CSS 3 es **display:inline-block**; esto le indica a nuestro section que todo lo que venga dentro del será visto como bloques y les añadimos el atributo **vertical-align:top** este nuevo atributo de CSS3 viene a sustituir al float del css anterior esto le indica a nuestro **section** que alinee todo verticalmente y que sea en la parte superior. Luego les indicamos que tome un tamaño de un 65% del section.

```
section#principal, aside{ background:#F1F1F1; border-radius:0.5em;
display: inline-block; margin:0.25em auto; max-width:960px;
min-height:200px; padding:0.25em; text-align:center;
vertical-align:top; width:65%; }
```

imagen 31

```
aside{ width:30%; }
```

imagen 32

Ahora solo nos falta darle estilo al article, este será sencillo solo tenemos que ponerle un color de fondo distinto a fondo actual para ellos les ponemos un **background:#000**; que corresponde al negro, luego un **border-radius** para las esquinas curvas y les ponemos un alto mínimo con el atributo **min-height**: de 140px; y un **padding**: de 0.25em con esto quedaría listo nuestro article.

```
article{ background:#011E30; border-radius: 0.5em;
min-height:140px; padding:0.25em; }
```

imagen 31



imagen 32



Si te fijas en el navegador reduciendo el tamaño de la ventana veras como todo se ajusta al tamaño de la ventana, esto lo obtuvimos porque trabajamos con los tamaños en (em) y no en (px) también porque los atributos **max-width** y **min-height**. Nuestro código completo serial:

```
index.html
<!DOCTYPE html/>
<html lang="es">
<head>
  <meta charset="utf-8" />    <title>Curso de HTML5 y CSS3    </
title>
  <link rel="stylesheet" href="stylo.css" />
</head>
<body>
  <header>
    <h1>Esta es la cabecera de nuestro sitio web</h1>
  </header>
  <nav>    Esta es nuestra botonera    </nav>
  <section id="contenedor">
    <section id="principal">
      Esto es una seccion
      <article>    Esto es un article    </article>
    </section>
    <aside>    Esto es un aside    </aside>
  </section>
```

imagen 33



```
<footer>
    Este es el pie de nuestra página
</footer>
</body>
</html>

stylo.css
*{margin: 0; padding: 0;}

@font-face { font-weight:normal; font-style:normal;}

body{ background: #FFF; color:#F60;
font-family:Helvetica, Verdana; font-size: 2em;}

h1{ color:#F60; margin:0.25em auto;
text-align:center; font-size:1.25em;
text-shadow: 5px 5px 10px rgba(255,255,255,0.5);}

header,nav,footer { background:#DEDEDE;
border-radius:0.5em; margin: 0.5em auto;
max-width:960px; padding:0.25em;
text-align:center; }

section#contenedor{ margin:0.2em auto;
```



```
max-width:960px;    padding:0;
text-align:center;  }

section#principal, aside{    background:#F1F1F1;
border-radius:0.5em;    display: inline-block;
margin:0.25em auto;    max-width:960px;
min-height:200px;    padding:0.25em;    text-align:center;
vertical-align:top;    width:65%;    }

article{    background:#000;    border-radius: 0.5em;
min-height:140px;    padding:0.25em; }

aside{    width:30%;}
```



Desempeño 1

Como ejercicios de esta clase, codificaremos los ejemplos expuestos en la misma.



Autoevaluación

- 1| ¿Cuál fue el primer navegador que introdujo CSS?
- 2| ¿Las hojas de estilo son sensibles a las mayúsculas y a las minúsculas?
- 3| ¿Es mejor usar la medida en píxel (px) o en puntos (pt)?
- 4| Investiga y aplica las siguientes propiedades: transition-property, transition-duration, transition-delay, transition-timing-function
- 5| Cuáles son las diferencias entre Box-shadow y drop-shadow?

Créditos

Imágenes

Encabezado: Image by Steve Buissinne from Pixabay

<https://pixabay.com/photos/pawn-chess-pieces-strategy-chess-2430046/>

Tipografía

Para este diseño se utilizó la tipografía *Source Sans Pro* diseñada por Paul D. Hunt. y *Space Mono* diseñada por Colophon Foundry Extraída de Google Fonts.

Si detectás un error del tipo que fuere (falta un punto, un acento, una palabra mal escrita, un error en código, etc.), por favor comunicate con nosotros a correcciones@issd.edu.ar e indicanos por cada error que detectes la página y el párrafo. Muchas gracias por tu aporte.